

Politecnico di Torino

Politecnico di Torino

Master of science program in CYBERSECURITY

A.a. 2025/2026

Graduation Session September 2026

An Experimental Framework for Evaluating Secure Deletion Techniques in Modern Digital Environments

Relatori:

Andrea Atzeni

Giuseppe Emiliano Vaciago

Candidati:

Imesha Ann Anjalie Ponnamperumage

Abstract

Within the branch of Anti-Forensics, secure deletion has emerged as a critical component in cybersecurity, regulatory compliance and digital Forensics. While standard deletion operations remove file-system references without erasing the underlying data, secure deletion techniques aim to ensure that data is made irrecoverable. This Thesis presents an experimental framework for evaluating the effectiveness, performance, and observability of secure deletion techniques in modern solid-state drives. The framework addresses the lack of standardized and reproducible evaluation methods by considering measurable factors such as post-deletion data recoverability, deletion latency, timing variability and verification limitations. The experimental study evaluates data recoverability after TRIM, conventional overwriting, and drive-level sanitization at different time intervals. The experiments also examine how filesystem journaling, overwrite operations under Flash Translation Layer remapping, workload intensity, drive fill level, and vendor-specific controller behaviour influence deletion outcomes. The results are analysed to identify the reliability, limitations, and security implications of current deletion mechanisms, and to determine how effectively they support verifiable and standards-aligned data sanitization. In addition to the technical evaluation, this thesis also explore the regulatory and legal implications of secure deletion practices, particularly in the context of data protection laws and individual privacy rights. By establishing a reproducible evaluation methodology, this work contributes a foundation for systematically assessing secure deletion techniques and bridging the gap between technical capabilities and regulatory compliance requirements.

The importance of secure deletion is highlighed in cases where sensitive information, such as personal data, financial records, or confidential business information, expected by users to be permanently removed, can be recovered by malicious actors or forensic investigators. This disconnect between user expectation and technical reality has led to a cascade of high-profile data breaches across public and private sectors. Further the widespread transition from HDDs to SSDs has resulted in a paradigm shift in data storage and management, introducing new challenges for secure deletion through out-of-place updates, wear levelling, garbage collection, and Flash Translation Layer abstraction. These mechanisms can preserve residual copies of deleted data even after deletion or overwriting operations have been performed.

Keywords : *secure data deletion; NAND flash memory; solid-state drives; Flash Translation Layer; data remanence; data sanitization; cryptographic erasure; block erasure; deletion verification; regulatory compliance.*

Acknowledgements

Disclosure of AI use

For the preparation of this thesis, AI tools were used as supplementary support to refine and improve text that had been independently written by the author, particularly with respect to grammar, clarity, readability, sentence structure, and academic style.

Below tools were also used to troubleshooting resources during the development and execution of the experimental setup. They were consulted to investigate technical issues involving scripts, Linux commands, software configuration, and experimental procedures, and to suggest possible explanations or solutions.

Suggestions provided by these tools were not accepted without verification and were tested and evaluated by the author before being incorporated into the experimental workflow.

- ChatGPT (OpenAI)
- Deepseek

Table of Contents

List of Tables VI

List of Figures VII

1 Introduction 1
- 1.1 Identified Research Gaps . . . 3
- 1.2 NAND flash storage architecture . . . 4
  - 1.2.1 Cell Structure and Memory Organization . . . 4
  - 1.2.2 Basic Operations and Timing . . . 5
  - 1.2.3 Write-Once Property and Out-of-Place Updates . . . 5
  - 1.2.4 Flash Translation Layer . . . 6
  - 1.2.5 Scrubbing: Overwriting at the Page Level . . . 7
  - 1.2.6 Partial Overwriting and Down-Bit Programming . . . 8
  - 1.2.7 The Communication Path . . . 8
- 1.3 Implications for Secure Deletion . . . 10
  - 1.3.1 Verification of Secure Deletion . . . 13
- 1.4 Native Drive-Level Sanitization . . . 14
  - 1.4.1 Key Native Commands by Interface . . . 15
  - 1.4.2 What is MEK Rotation . . . 17

2 Comparative Analysis of Existing Literature 19
- 2.1 A Taxonomy of Anti-Forensic Techniques . . . 19
- 2.2 A Comparative Analysis of Secure Deletion Techniques . . . 20
  - 2.2.1 A Layer-Based Taxonomy of Secure Deletion . . . 20
  - 2.2.2 Comparative Evaluation of Secure Deletion Techniques . . . 26
  - 2.2.3 Comparison Matrix . . . 29
- 2.3 Synthesized Observations from the Literature . . . 44
- 2.4 Empirical Evaluation Approaches in the Literature . . . 47
- 2.5 Legal and Ethical Dimensions . . . 47
  - 2.5.1 GDPR and the Right to Erasure . . . 47
  - 2.5.2 Disposal and reuse of storage media . . . 49

2.5.3 Accountability and evidencing deletion 50
2.5.4 Technical Challenges to the Right to Be Forgotten 51
2.5.5 Digital Evidence and Cross-Border Procedure 51
2.5.6 Comparative Regulatory Landscape 52

3 Experimental Framework and Methodology 54
3.1 Research Objectives 55
3.1.1 Research Questions 55
3.2 Experimental Environment 56
3.3 Experiments Conducted 58
3.3.1 Experiment 1: TRIM and Filesystem-Level Data Recoverability 58
3.3.2 Experiment 2: Data Recovery After TRIM 59
3.3.3 Experiment 3: Testing Recoverability at different fill levels and times intervals 63
3.3.4 Experiment 4: Cryptographic Erase Attempt 65

4 Experimental Results and Analysis 66
4.1 Results Analysis 66
4.1.1 Experiment 1: TRIM and Filesystem-Level Data Recoverability 66
4.1.2 Experiment 2: Data Recovery After TRIM 71
4.1.3 Experiment 3:Testing Recoverability at different fill levels and times intervals 78
4.1.4 Experiment 4: Cryptographic Erase Attempt 80

5 Discussion of Findings 82
5.1 Summary of Main Findings 82
5.2 Limitations of the Study 85

6 Conclusions and Future Work 86
6.1 Conclusions 86
6.2 Future Work 86

A Appendix A 88
A.1 Device Specifications 88
A.1.1 Host system 88
A.1.2 Test Drive 1: NVMe SSD 88
A.1.3 Test Drive 2: SATA SSD 90
A.2 Experiment 1 92
A.3 Experiment 2 93
A.3.1 Experimental Setup 93
A.3.2 Trim-on and Trim-off Conditions 94
A.3.3 Data Generation Script 94

A.3.4 Command List 95
A.4 Experiment 3: Full Script 99

List of Tables

1.1 Comparison of NIST SP 800-88 Sanitization Levels 15
1.2 The Key Hierarchy Within an NVMe SED 17

2.1 Summary of Anti-Forensic Techniques 20
2.2 Secure Deletion Approaches by System Layer 25
2.3 Comprehensive Comparison of Secure Deletion Approaches for Flash Storage 30

3.1 Dataset Allocation for the TRIM Recovery Experiment 61

4.1 Data Recovery Results After TRIM on NVMe SSD 72
4.2 Data Recovery Results After TRIM on SATA SSD 74
4.3 Recovery Results 79
4.4 Recovery Time Results 80
4.5 Results of ATA Sanitization Command Attempts 80

List of Figures

4.1 Mean Recovery on NVMe SSD 72
4.2 Mean Recovery on SATA SSD 74
4.3 Standard Deviation of Recovery on NVMe and SATA SSDs 77

Chapter 1

Introduction

The task of deleting data and ensuring that they remain irrecoverable is known as Secure Data deletion. It has currently become one of the most pressing concerns in the cybersecurity, regulatory compliance, and digital forensics domains. While there exists physical means of making data irrecoverable in the physical world where confidential documents are shredded, and storage media containing classified material are physically destroyed, in the digital domain this is significantly complex. The standard delete operation on modern systems only implements file unlinking, which only changes the file system metadata while full contents will still remain intact in the system but users assume that all the content has been permanently erased. [**reardon2013sok**]

The criticality of this problem has been addressed by two intersecting forces. First, regulatory frameworks including GDPR right to erasure, NIST SP 800-88 Rev. 1, and IEEE 2883-2022 have mandated demonstrative and verifiable data removal evidences at defined sanitization levels; Clear, Purge and Destroy, with specific assurance against data recovery. Next, the mainstream transition from the usage of Hard Disk Drives (HDDs) to Solid-State Drives (SSDs) has led to a fundamental alteration of the storage landscape resulting in decades of secure deletion practices being inadequate.[**aronsson2025secure**] The problem of secure deletion not only occurs in the technical domain but also spans into the legal and ethical domains. On the legal front, regulations such as the GDPR, the California Consumer Privacy Act (CCPA), and emerging AI governance frameworks impose obligations for data erasure that existing technical mechanisms struggle to satisfy.

Taking into account the criticality of the problem, a Study conducted by two MIT graduate students in 2003 by purchasing 158 used hard drives from eBay found that 117 (74%) drives contained recoverable data, 28 (17%) had fully installed operating systems with user data requiring no special effort to recover, only 12 drives (8%) had been properly sanitized. Recovered data included medical records, love letters, corporate financial records, and over 5,000 credit card numbers [**MITstudy2003**].

A near-identical experiment was conducted by Blancco Technology Group in 2018 with 159 used drives across the U.S., U.K., Germany, and Finland. Every seller insisted the drives had been "wiped." Researchers recovered passport scans, spreadsheets full of customer data, and intimate photographs. [**NetGuardia2026**].

As per the 2024 Verizon Data Breach Investigations Report, 68% of data breaches involve non-malicious human error—including failure to properly delete sensitive information from decommissioned devices [**Verizon2024DBIR**]. Separately, Blancco Technology Group's annual data sanitization report found that 42% of used drives sold on secondary markets contain recoverable personally identifiable information (PII), with 15% containing corporate financial data [**Blancco2026**]. The public sector contributes some of the earlist cases, In 2002, the Pennsylvania Department of Labor and Industry sold computers to local resellers without sanitizing the drives. The machines contained "thousands of files of information about state employees" that had never been removed [**MITstudy2003**]. More recently, in 2026, Cornwall Council in the UK suffered a redaction failure when it sent PDF documents that appeared redacted but revealed the full personal details of 10 complainants—names, home addresses, emails, and phone numbers—when opened. The failure mechanism was using visual overlays, a redaction approach that does not securely delete the underlying data instead of applying permanent redaction with sanitization [**ComplianceHub2026**].

Within the financial Sector, Between 2015 and 2022, Morgan Stanley Smith Barney (MSSB) committed a series of secure deletion failures that became one of the most expensive examples of improper data disposal on record. In 2015, MSSB decommissioned thousands of hard drives and servers by hiring a company which had no prior experience or expertise in proper data destruction. These hard drives and server were later resold on an internet auction site while containing the PII of 15 million customers. [**Goodin2022**]. In a separate incident, 42 servers went missing during a hardware refresh program. All potentially contained unencrypted customer data. The servers had encryption capability, but the firm never activated the encryption software. Recovered drives contained at least 140,000 pieces of customer PII on 13 out of 14 hard drives examined forensically. The SEC stated the "vast majority" of devices were never recovered [**SEC2022**]. The company ended up paying over $155 million in fines and settlements related to these incidents [**CybersecurityDive2022**].

The common thread across every case is the same: standard deletion does not remove data. Operating systems typically only remove file system pointers, leaving the underlying data intact on the storage medium. On SSDs, the problem compounds because the Flash Translation Layer performs out-of-place updates, wear leveling, and garbage collection that can silently preserve copies even after overwrite attempts. Whether the mechanism is a visual redaction overlay, a missing encryption activation, an unverified third-party promise, or a simple failure to wipe

before resale, the result is identical: data thought to be gone remains recoverable.

1.1 Identified Research Gaps

Secure deletion in modern digital environments is a multi-layered problem spanning storage architecture, cryptographic key management, regulatory compliance, and empirical verification. Existing literature has produced a rich taxonomy of techniques overwriting, encryption-based, erasure-based, and integrated approaches each with distinct trade-offs. The central research gap addressed by this thesis is the absence of a standardized and reproducible framework for evaluating secure deletion techniques across modern storage environments. Existing research has examined individual deletion mechanisms, file systems, storage devices, or controller-level techniques, but the experimental methods, hardware configurations, evaluation criteria, and verification procedures vary considerably. This variation limits the comparability of results and makes it difficult to determine whether a deletion method is effective across different devices and operating conditions. Existing evaluations are also insufficient because successful completion of a deletion command does not necessarily demonstrate physical data removal. TRIM, for example, informs the controller that specific logical blocks are no longer required, but it does not provide the host with a definitive indication of when or whether the corresponding physical pages have been erased. The interval between logical invalidation and physical erasure, referred to in this thesis as zombie time and explained in detail in Chapter 2, may depend on workload pressure, garbage-collection behaviour, over-provisioning, and the controller's victim-block selection policy.

To address these limitations, this thesis proposes an experimental framework based on observable and reproducible evidence. The framework evaluates data recoverability after TRIM, conventional overwriting, and drive-level sanitization operations. For TRIM experiments, recoverability is examined at different time intervals defined by observable events, such as command-completion signals, failed or successful recovery attempts, and available device-status information. The framework also examines how workload conditions, device characteristics, file-system journaling, and overwrite operations affected by FTL remapping influence the persistence of deleted data. In addition to recoverability, the framework evaluates the latency and variability of sanitization operations. Experiments are conducted at different drive fill levels to determine whether a partially used drive behaves differently from a heavily used drive. The analysis considers both average execution time and unpredictability between repeated measurements, since vendor-specific controller behaviour may produce substantially different results even for similar storage devices. The proposed framework differs from existing evaluations in several ways. First, it evaluates deletion through post-operation recoverability

rather than assuming that a successful command guarantees physical erasure. Second, it combines security, performance, workload, device-state, and verification dimensions within a single evaluation structure. Third, it treats drive fill level, controller implementation, and vendor behaviour as experimental variables rather than uncontrolled background conditions. The objective of this thesis is therefore not to identify a universally effective deletion command. Instead, it is to establish a systematic basis for comparing deletion mechanisms, defining their observable security guarantees, identifying their limitations, and assessing their suitability for reliable and standards-aligned data sanitization. The following sections first describe NAND flash architecture and its implications for secure deletion, then review existing techniques and evaluation methods, before presenting the proposed framework, experimental results, and analysis. And finally, the thesis concludes with a discussion of the findings and directions for future research.

1.2 NAND flash storage architecture

NAND flash memory which was introduced by Toshiba in 1988, is a type of non-volatile semiconductor memory that has become the dominant storage technology for solid-state drives (SSDs), USB flash drives, and mobile devices [**aronsson2025secure**]. Unlike NOR flash, which supports random access and high read performance at smaller capacities, NAND flash offers significantly higher storage density and faster write/erase operations at a lower cost, making it ideal for bulk data storage [**homaidi2009data**]

1.2.1 Cell Structure and Memory Organization

At the physical level, each NAND flash memory cell is a floating-gate transistor. Data is stored by injecting electrical charge into the floating gate, which is electrically isolated and can retain charge without power. A charged floating gate (programmed state) represents a logical "0," while an uncharged gate (erased state) represents a logical "1" [**homaidi2009data**]. The amount of charge determines the cell's threshold voltage (Vt), which is read and interpreted as one or more bits depending on the cell type:Single-level cell (SLC) flash stores one bit per cell, whereas multi-level cell (MLC) stores two bits, triple-level cell (TLC) stores three bits, and quad-level cell (QLC) stores four bits. Each increase in bit density reduces cost per gigabyte at the expense of reduced endurance and lower performance [**aronsson2025secure**].

Memory cells are arranged hierarchically. Cells connected to the same word line form a page, which is the smallest unit for read and program (write) operations. Pages are grouped into blocks, the smallest unit for erase operations. Blocks are organized into planes, multiple planes reside on a single die, and multiple dies are

packaged into a NAND chip. Typical page sizes range from 2 KB to 16 KB, and blocks typically contain 64 to 256 pages[**aronsson2025secure**]. Each page also includes a spare or out-of-band (OOB) area used to store error-correcting codes (ECC) and metadata [**duplicateschen2022**]. During a normal program operation, charge is injected into the floating gate using Fowler-Nordheim tunneling, shifting the Vt to a target distribution window. A read operation applies a reference voltage and senses whether current flows, revealing the cell's programmed state. The erase operation removes charge by applying a high voltage across the transistor, returning the cell to its erased (lowest Vt) state.

1.2.2 Basic Operations and Timing

The three fundamental operations on NAND flash are read, program, and erase. Read and program operate at page granularity, while erase operates at block granularity. Their latencies differ by orders of magnitude: a page read takes approximately 20 to 60 μs, a page program takes 200 to 500 μs, and a block erase takes 1.5 to 5 ms [**aronsson2025secure**, **duplicateschen2022**]. An erase-before-write architecture means that updating even a single byte requires eventually erasing an entire block, which the Flash Translation Layer (FTL) works around using out-of-place updates. The FTL abstracts these physical details behind a standard block-device interface, mapping logical block addresses (LBAs) to physical NAND pages [**onarlioglu2018_3**]. This abstraction allows traditional file systems (ext4, NTFS, HFS+) to remain compatible, but it also hides the true physical state of the data from the host [**aronsson2025secure**].

1.2.3 Write-Once Property and Out-of-Place Updates

NAND flash has a fundamental constraint: a page cannot be overwritten in place. To write new data to a page, the entire encompassing block must first be erased, which resets all cells in that block to the "1" (erased) state [**aronsson2025secure**] Because erasing a block is an expensive operation that destroys all data within it, SSDs do not perform in-place updates. Instead, they use out-of-place updates: modified data is written to an available free page elsewhere, and the old page is marked as invalid [**zhang2018ensuring**]. Pages containing current data are termed valid pages, while those holding superseded data are invalid pages. This write-once property is the root cause of many of the secure deletion challenges discussed in this review.

1.2.4 Flash Translation Layer

To maintain compatibility with traditional block-based file systems (e.g., ext4, NTFS), SSDs incorporate a Flash Translation Layer (FTL) that runs on the device controller [**aronsson2025secure**]. The FTL maps logical block addresses (LBAs) issued by the operating system to physical flash addresses. This abstraction hides the complexity of NAND flash from the host and enables the SSD to manage internal operations transparently [**duplicateschen2022**].

The FTL implements several critical management functions:

Wear leveling distributes program/erase (P/E) cycles evenly across all blocks to extend device lifespan. Each NAND flash block has a finite endurance typically 100,000 cycles for SLC and 10,000 to 100,000 for MLC/TLC after which the block becomes unreliable. Dynamic wear leveling distributes writes among free blocks, while static wear leveling periodically relocates infrequently accessed (cold) data to prevent some blocks from remaining idle while others wear out [**homaidi2009data**].

Garbage collection reclaims space occupied by invalid pages. When the number of free pages falls below a threshold, the FTL selects a victim block with a high proportion of invalid pages, copies any remaining valid pages to a free block, and then erases the victim block [**aronsson2025secure**, **duplicateschen2022**]. This process introduces write amplification, where a single host write triggers multiple internal writes due to data migration.

Bad block management tracks blocks that have become defective through wear or manufacturing defects. These blocks are recorded in a bad block table and excluded from further use, with any valid data relocated to healthy blocks [**duplicateschen2022**, **homaidi2009data**].

Over-provisioning reserves a portion of the total flash capacity as spare area to improve performance and endurance. This over-provisioned space is used for wear leveling, garbage collection, and bad block management. [**aronsson2025secure**, **ahn2021forensics**]

Error correction is essential because NAND flash cells are susceptible to read disturb, program disturb, charge leakage, and retention errors. The FTL computes ECC data stored in each page's OOB area to detect and correct bit errors[**zhang2018ensuring**].

An SSD packages multiple NAND chips with a controller that manages the FTL, host interface, and error correction. The storage architecture spans several layers: the host interface (e.g., SATA or NVMe) communicates with the FTL, which translates LBAs to physical addresses and coordinates access across multiple channels, packages, and dies in parallel to maximize throughput. To overcome the density scaling limits of planar (2D) NAND, manufacturers have adopted 3D NAND technology, which stacks memory cells vertically in layers [**aronsson2025secure**].

3D NAND achieves higher storage density without shrinking individual cell geometries, which improves endurance and reduces interference between adjacent cells. However, the vertical architecture introduces more complex disturb characteristics and new challenges for secure deletion, as erasing data in one layer can disturb adjacent layers [**ahn2021forensics**].

1.2.5 Scrubbing: Overwriting at the Page Level

Since page-level erase is physically impossible, scrubbing was proposed by Wei et al. in 2011 as a way to destroy data on individual programmed pages without block erasure. Scrubbing works by re-programming all remaining 1 bits on a page to 0, effectively overwriting the entire page with zeros. This is possible because NAND flash does allow programming individual bits from 1 to 0; the reverse transition (0 to 1) is what requires a block erase. The mechanism uses incremental step pulse programming (ISPP), which iteratively applies programming pulses and verifies the cell Vt until it reaches the target state [**aronsson2025secure**, **3dnandwang2018**].

Scrubbing comes with a serious reliability cost called cell disturbance. The act of reprogramming one cell unintentionally shifts the Vt of neighboring cells due to parasitic coupling through the word lines and bit lines. In 2D NAND, programming a page disturbs the two immediately adjacent pages. In 3D NAND, where flash cells are stacked vertically, as many as four adjacent pages can be disturbed, and the problem grows more severe with each additional layer. The increased bit error rate (BER) from scrubbing disturbance may trigger error correction failures, requiring the data on affected adjacent pages to be migrated before scrubbing can safely proceed.

The concept of a scrub budget was introduced to limit how many times a given block can be scrubbed before exhibiting significant data error risk. Once the budget is exceeded, secure deletion for that block must fall back to a full block erase or garbage collection. Determining the correct budget requires experimental BER analysis and is pattern-dependent and wear-cycle-dependent, making it difficult to set a universal safe threshold[**antiforensics2016**, **zhang2018ensuring**].

A full scrubbed page is uniformly zeros, which itself signals to a forensic examiner that a deletion occurred at that location. Partial scrubbing addresses this by modifying only a subset of bits in the page or block, avoiding the tell-tale all-zero pattern. This is a form of anti-forensic deletion that blends sanitized data with the appearance of normal programmed content.

More critically, Hasan and Ray demonstrated in 2020 that scrubbing is partially reversible[**Fukami2023**]. By performing analog reads that measure the precise threshold voltage distribution (rather than a single binary interpretation), an attacker can distinguish between a cell that was programmed to a 1 and then

scrubbed to 0, and a cell that was originally programmed to 0. The residual Vt difference, though small, is detectable with precision sense amplifiers. Analog sanitization was proposed as a remediation: instead of a single binary scrubbing pass, the page is reprogrammed to a Vt distribution that is statistically indistinguishable from a native 0-programmed page[**ahn2022security**].

1.2.6 Partial Overwriting and Down-Bit Programming

Multiple finer-grained overwriting schemes operate at sub-page granularity by exploiting the fact that MLC and TLC cells have multiple threshold voltage states. Partial overwriting programs random data into only some of the Vt states of a multi-bit cell, destroying the original data without fully programming the cell to the highest state. Down-bit programming takes this further by programming a page originally stored in a 3-bit-per-cell (TLC) mode into a 1-bit-per-cell (SLC) mode, effectively compacting and destroying the original data in a single operation. The deletion pulse application scheme applies a series of erase-like pulses to a programmed page, shifting its Vt distribution without performing a full block erase. These schemes cause lower program disturbance than conventional scrubbing because they do not push cells to the highest Vt state, but they also provide less security assurance since some Vt residue may remain [**ahn2019schemes**, **ahn2022security**].

1.2.7 The Communication Path

The operating system does not have direct knowledge of the physical flash layout. After a file is deleted, the file system deallocates the logical blocks by removing their entries from the file system index (inode table, MFT, etc.), but the actual data remains intact on the NAND cells. The host cannot simply overwrite the physical location because the FTL may have remapped those logical addresses to any physical page on the drive. There are three main channels through which the OS conveys block-deallocation information to the storage device

TRIM : The TRIM command is an ATA command that allows the file system to inform the SSD controller about which LBAs no longer store valid data. When a file is deleted, the OS issues a TRIM command for the LBAs covered by that file, and the controller adds those blocks to an internal queue of blocks to be blanked. The TRIM command was not designed for secure deletion but rather as an efficiency optimization for flash: without it, the FTL cannot distinguish logically discarded blocks from valid data and must assume that the only deletable block on the physical medium is the one being overwritten, eventually causing a thrashing effect. TRIM operates through the ATA DATA SET MANAGEMENT command and is supported by most modern operating systems (Windows 7+, Linux 2.6.28+,

macOS) and file systems (NTFS, ext4, HFS+). Importantly, the TRIM command provides no false negatives: it indicates every time a block is discarded. However, it is an asynchronous notification: the controller is free to schedule the actual physical erasure at its own discretion, and the OS receives no confirmation of when or whether the erasure happened. TRIM is a suitable command for deleting specific files, but the FTL only uses the information to mark address mappings as invalid [**reardon2013sok**, **trimwearlevelling2019**].

TrueErase : Diesburg et al. proposed TrueErase as a refinement that provides similar information to TRIM but only for blocks belonging to files specifically marked as sensitive. TrueErase adds a new communication channel between the file system and the device driver that forwards deallocation information selectively. Device drivers are modified to implement immediate secure deletion when provided with this notification, avoiding the performance cost of blanket TRIM for all deleted blocks [**reardon2013sok**].

Out-of-place updates as implicit notification : When the host writes new data to an LBA that already has a physical mapping, the FTL performs an out-of-place update: it allocates a new physical page, writes the new data there, and marks the old physical page as invalid. This gives the FTL indirect evidence that the old data at that LBA is no longer needed. However, this mechanism only works for blocks that are explicitly overwritten, not for blocks that are simply deallocated by file deletion or truncation [**antiforensics2016**].

The FTL performs the address mapping function that translates the logical addresses used by the host into the physical addresses on the NAND flash memory. When the host issues a deletion or overwrite request, the FTL invalidates the address mapping information linking the logical address to the previous physical address. For deletion, the FTL receives the file's LBA range through the TRIM command and marks the corresponding address mappings as invalid [**antiforensics2016**].

Crucially, the invalidation step is purely a metadata operation. The FTL terminates the host's request immediately after updating the mapping table, and the actual data on the NAND cells is left entirely untouched. The flash controller does not erase, overwrite, or otherwise modify the physical pages at this point. The data persists in an undamaged state on the NAND cells, fully recoverable through low-level access techniques. The only change is that the FTL will never again return that physical page in response to a read of that LBA. This design is intentional: erasing a block is time-consuming (on the order of milliseconds per block), and performing it eagerly on every deallocation would destroy performance[**duplicateschen2022**].

1.3 Implications for Secure Deletion

The architectural features described above create fundamental obstacles to secure deletion. Out-of-place updates cause multiple copies of data to persist across the device as stale pages are left behind. Garbage collection and wear leveling can relocate data transparently, making it impossible for host-level software to predict where data physically resides. Over-provisioned areas are inaccessible to the operating system, yet may retain residual copies of deleted data. These characteristics mean that a conventional delete command at the file-system level does not remove the underlying data from the physical medium, and techniques that are effective on HDDs such as overwriting are ineffective on NAND flash [**aronsson2025secure**].

Garbage Collection and the Zombie Time

Garbage collection is the FTL process that physically reclaims NAND blocks containing invalid pages. Its canonical procedure has three steps: selecting a victim block with the largest number of invalid pages, copying any remaining valid pages from the victim block to a free block, and then erasing the victim block [**duplicateschen2022**]. The erase operation makes the block available for future writes. Free blocks are managed from a pool, and modern SSDs are typically over-provisioned with 25% or more spare NAND capacity to absorb write pressure and give garbage collection time to keep up [**onarlioglu2018_3**].

The zombie time is defined as the interval between the point when the FTL invalidates the address mapping of a file's pages and the point when garbage collection physically erases those pages from the NAND cells [**antiforensics2016**]. During this window, the data remains fully intact and recoverable, even though the file system and the host consider it deleted. The zombie time has no fixed duration: it depends on the storage usage pattern, the prevalence of free blocks in the over-provisioning pool, the FTL's victim selection policy, and whether TRIM commands were issued for the deallocated blocks.

The fundamental cause of the zombie time is that the FTL is designed to minimize garbage collection frequency, since garbage collection degrades I/O response by introducing read-copy-erase cycles that add latency. The FTL only triggers garbage collection under two conditions. In the foreground (normal garbage collection), garbage collection is triggered when the free-block pool drops below a threshold, meaning the device has no clean blocks available to service incoming writes. n the background, some FTL implementations opportunistically erase stale blocks during idle time. The foreground garbage collection event is driven entirely by write pressure: if the host is mostly reading and the free-block pool is adequate, garbage collection may not occur for long periods. There is a compounding factor: even when

normal garbage collection is triggered, the zombie time for a specific file depends on whether the blocks containing its pages are selected as victim blocks. The FTL selects the victim with the highest count of invalid pages [**antiforensics2016**], so a block containing only a few invalid pages is unlikely to be chosen until many pages in that block have accumulated. If a block contains mostly valid pages plus one or two invalidated pages from a deleted file, that block will sit untouched indefinitely while the deleted data remains recoverable.

The Zombie-Kill (ZK) TRIM scheme, proposed by Kwak et al., introduces a compulsory garbage collection mechanism that directly limits the zombie time, it operates in three modes that trade deletion speed against I/O performance [**antiforensics2016**]. *Normal mode* defers all physical erasure to the FTL's standard garbage collection policy (normal garbage collection). The TRIM handling process is the same as standard file deletion: the address mapping is invalidated, and the target pages are erased only when the free-block pool runs low and the blocks happen to be selected as victims. This mode has the longest zombie time, and in workloads with sufficient free blocks, the zombie time becomes effectively unbounded: deleted data may persist indefinitely because garbage collection is never triggered. *Immediate mode* erases the data using the compulsory garbage collection upon receiving the TRIM command. As soon as the TRIM command arrives, the FTL invalidates the address mappings and then immediately performs compulsory garbage collection on the blocks containing the target pages. This mode limits the zombie time to the execution time of the compulsory garbage collection itself, which is determined by the number of blocks containing target pages and the number of valid pages that must be copied out of each victim block. In the experiments by Kwak et al., immediate mode achieved the smallest zombie time in all cases, but the I/O response time was 68 times larger on average than normal mode because the compulsory garbage collection introduces processing delays even when free blocks are abundant. *Suspension mode* balances the trade-off. After receiving the TRIM command, the FTL saves the invalidation time of each deleted file's pages to a TRIM command table. The FTL then monitors the suspension time of each file by comparing the current time against the invalidation time. Until the suspension time exceeds a configurable maximum, target pages can only be erased by the normal garbage collection policy. If target pages remain unerased after the maximum suspension time, the blocks containing them are erased by the compulsory garbage collection. This mode limits the zombie time to the sum of the maximum suspension time and the compulsory garbage collection execution time, while producing a smaller I/O penalty than immediate mode. Fewer compulsory garbage collection events occur because target pages accumulate during the suspension window, and blocks with a higher proportion of invalid pages are selected, making each garbage collection event more efficient [**antiforensics2016**].

The zombie time is workload-dependent with significant forensic implications.

In write-heavy workloads, the free-block pool drains quickly and normal garbage collection is triggered frequently, reducing the zombie time. In read-heavy workloads, the free-block pool remains ample, and normal garbage collection may never activate the blocks containing deleted pages. Under these conditions, the zombie time can be effectively infinite: the deleted data persists until the device is eventually subjected to a write- intensive workload that forces garbage collection. Compounding this, the distribution of invalid pages across blocks affects whether a given deleted file's pages are reclaimed. The FTL selects the victim with the largest number of invalid pages, so blocks where the deleted file's pages are scattered across many blocks (each containing few invalid pages) experience longer zombie times than blocks with concentrated invalid pages.

TRIM Implementation Variance Across Drives

: Black-box testing has revealed significant variation in how different SSDs implement TRIM and garbage collection. On the Corsair F60, TRIM triggered within 1-10 seconds of a deletion. On other drives, TRIM behavior depended on the percentage of allocated space. Some drives did not erase all TRIM-marked blocks but rather a subset based on internal drive policies, meaning that even after TRIM and GC, some deleted blocks remained unerased. Importantly, garbage collection can operate independently of the host: it can be triggered whenever the disk is powered on, even if no write commands are issued, rendering write blockers and other forensic precautions useless. This means the forensic examiner cannot assume that data persists in a known state across reboots. Most modern operating systems and their default file systems have built-in support for the ATA TRIM command, but the implementation of TRIM varies from controllers in SSDs to kernels in operating systems. [**Bonetti2014**, **trimwearlevelling2019**]

FTL Internals and its Forensic Impact

Wear leveling distributes program-erase (P/E) cycles evenly across all blocks to prevent premature wear-out of frequently written blocks. Dynamic wear leveling operates on newly allocated blocks, while static wear leveling also moves cold (infrequently rewritten) data to ensure even wear on otherwise idle blocks. Every data migration performed by wear leveling creates opportunities for data duplication: a page may be copied to a new physical location while the old location is not immediately erased, leaving recoverable copies that neither the file system nor the OS is aware of. The write amplification factor (WAF) measures the ratio of physical writes to logical writes. A WAF of 3 means that every logical write from the host causes 3 physical writes due to garbage collection and wear leveling overhead. Each extra write represents a potential duplicate of the original data,

and each duplicate must be separately sanitized for secure deletion to be effective. [**duplicateschen2022**, **onarlioglu2018_3**]

The copy-back program operation is an internal NAND flash operation that moves data from one page to another without transferring data to the controller. It is used by FTLs during garbage collection and wear leveling to reduce latency. Ahn and Lee revealed that the copy-back program operation leaves unmanaged original data in the source page, which becomes a major forensic target. The source page may contain the original data even after the copy-back completes, and because the copy-back is a low-level operation invisible to the host, no higher-layer deletion mechanism can target it. Secure copy-back programming adds an enhanced secure program operation after the conventional copy-back. This can take two forms: programming random data into the source page (destroying the stored data) or applying a series of deletion pulses to the source page to shift its Vt distribution. To minimize disturbance to adjacent victim cells, the controller can first read victim cell group information from the adjacent word line and selectively inhibit programming on cells that would cause errors. [**ahn2021forensics**]

FTL operations systematically create multiple copies of data that persist after the primary copy is deleted. Garbage collection copies valid pages from victim blocks to free blocks before erasing, leaving the original data in unerased cells that may remain in the free block pool. Wear leveling similarly migrates data, and the old physical location is not always erased immediately. Bad block management copies data away from failing blocks, creating yet another duplicate. The RedFlash scheme addresses this by maintaining a duplicate chain in the out-of-band (OOB) area of each flash page. The OOB area is typically reserved for ECC parity and bad block markers, the RedFlash was repurposed to store metadata linking each page to all other pages that contain copies of the same logical data. When a deletion command is issued, the OOB chain enables backtracking so that every duplicate is located and erased, not just the primary copy [**duplicateschen2022**].

Encryption-based deletion alone does not eliminate structural artifacts. Chen and Chen showed that the physical location of data on flash storage can itself leak information even when the data content is encrypted or destroyed. For example, knowing that certain blocks are allocated to specific applications or user profiles reveals usage patterns. When a block is sanitized while surrounding blocks remain intact, the spatial pattern of erasures implicitly confirms which data the user considered sensitive. Sanitizing data content without addressing structural context leaves a recoverable forensic profile [**aronsson2025secure**].

1.3.1 Verification of Secure Deletion

A persistent gap in secure deletion is the inability to independently confirm that data has been physically destroyed. Ahn and Lee proposed an in-band verification

technique based on cell count information. After a secure deletion operation, the NAND cells in the sanitized area exhibit a characteristic Vt distribution that differs from normal programmed data. Under normal program conditions, the number of cells in each Vt state is approximately uniformly distributed. After a scrubbing or block erase, the distribution shifts to a non-uniform pattern. A simple read operation can count cells per distribution and compare against a reference threshold, determining pass or fail without comparing personal data directly. This verification can be performed in real time immediately after secure deletion and requires no structural changes to existing NAND flash memory. For higher assurance, the verification scheme can use XOR operations between the data sensed from the sanitized area and the original personal data, with pass/fail determination accounting for the ECC correction capability. If the residual data is within the correctable bit error range, deletion is considered successful. [**ahn2021forensics**]

Modern SSDs can vary the size of the over-provisioning (OP) area in real time to optimize performance, with OP areas reaching up to 50% of total NAND capacity. Changing the OP area size generates an invalid data region: when OP shrinks, blocks that were previously reserved become accessible as user space, and the data they contain has never been sanitized. A forensic attacker can exploit this by manipulating the OP area allocation to expose blocks that still hold residual data. Even if the user data is encrypted, an attacker with sufficient resources could read the encrypted data from the invalidated area and attempt decryption offline. This makes deletion verification for dynamically allocated OP areas an essential but largely unaddressed requirement. Some proposed solutions include pseudo-erase operations that combine partial programming with lightweight verification, trading full sanitization for real-time performance [**ahn2021forensics**].

Achieving secure deletion in modern systems is fundamentally challenging because storage architectures are organized into multiple layers application, file system, block device, and physical medium each of which can retain residual copies of data [**duplicateschen2022**]. Performing deletion at the application layer, for example, only removes the logical reference; the underlying data remains in the file system and may persist on the physical media indefinitely [**zhang2018ensuring**].

1.4 Native Drive-Level Sanitization

Native drive-level sanitization refers to built-in hardware/firmware commands defined by storage interface standards (ATA, NVMe, SCSI) that instruct a drive's controller to erase all user data on the physical medium. Unlike software-based overwriting tools that operate through the file system, these commands execute inside the drive's controller and can potentially reach areas the OS cannot access — including over-provisioned zones and retired blocks [**antiforensics2016**].

The three major standards; NIST SP 800-88 Rev. 1 (2014), IEEE 2883-2022, and NATO AEDP-3 use a three tier classification of sanitization methods: Clear, Purge, and Destruct. [**aronsson2025secure**].

Table 1.1: Comparison of NIST SP 800-88 Sanitization Levels

Level	Description	Effectiveness	Reusability
Clear	Logical sanitization that removes user-accessible data only. Example: ATA Secure Erase, NVMe Format without CRYPTO flag.	Low — residual data may remain in hidden areas	Storage can be reused
Purge	Advanced sanitization making recovery infeasible even under lab conditions. Example: ATA Enhanced Secure Erase, NVMe Secure Erase (CRYPTO flag enabled), Cryptographic Erase.	High	Storage can be reused
Destruct	Physical destruction of the media.	Complete	Device cannot be reused

1.4.1 Key Native Commands by Interface

ATA/ATAPI

- `SECURITY ERASE PREPARE` : prerequisite that unlocks the drive for erasure.
- `SECURITY ERASE UNIT` : overwrites every NAND cell with zeros or ones. Operates at the factory-reset level and covers reserved areas (over-provision and retired blocks) that the host cannot normally address.
- `ENHANCED SECURITY ERASE` : vendor-defined extension that may erase more thoroughly (e.g., SanDisk calls this "sanitize").

- `SANITIZE DEVICE` : a newer, more flexible command supporting three modes: crypto scramble, overwrite, and block erase.
- `DATA SET MANAGEMENT` : not a sanitization command, but marks logical blocks as free so the controller's garbage collection can reclaim them over time.

NVMe

- `Format NVM` : reformats the namespace. Without the CRYPTO flag it is a Clear-level operation; with the CRYPTO flag enabled it reaches Purge level.
- `Sanitize` : supports Block Erase, Overwrite, and Crypto Erase modes. The `nvme sanitize-log` command reports progress and completion status.

SCSI

- `SANITIZE` : functionally similar to the ATA SANITIZE DEVICE command, supporting block erase, overwrite, and cryptographic erase options.

As stated above there are two principle purge mechanisms: block erase and cryptographic erase.

Block Erase : The controller issues the NAND flash erase command for every block on the device, forcing each block to its unprogrammed (erased) state. This is inherently slow — a 900 GB NVMe drive can take significant time — and the time estimates reported by tools are often inaccurate. Critically, block erase behavior is governed by usage-aware controller logic that may compromise the guarantee of full sanitization; studies found that the controller sometimes returns all-zero data during verification without having physically erased every cell, suggesting internal masking rather than true physical deletion. [**aronsson2025secure**]

Cryptographic Erase : On self-encrypting drives (those supporting TCG Opal SSC or similar), all user data is encrypted by a media encryption key stored in the controller. Cryptographic erase works by scrambling or deleting that key, making the ciphertext on the medium permanently undecryptable. This is nearly instantaneous and does not wear the NAND cells. The flip side: security rests entirely on the vendor's key management. If the key is not fully destroyed, if a copy persists in a controller cache, or if there is a backdoor, the protection fails. [**aronsson2025secure**]

As stated above,NVMe's Sanitize command supports Cryptographic Erase as one of its three modes. Cryptographic erasure renders data irrecoverable by destroying the media encryption key (MEK) rather than overwriting physical cells and n a self-encrypting drive, all user data is encrypted by a MEK that never leaves the controller's secure boundary. An NVMe self-encrypting drive complying with TCG Opal 2.0 maintains multiple keys at different privilege levels and the MEK is the root of every data confidentiality guarantee. All user data written to the NAND

medium is ciphertext under this single key, encrypted and decrypted transparently by the hardware encryption engine in the drive controller [**aronsson2025secure**].

Table 1.2: The Key Hierarchy Within an NVMe SED

Key	Role	Storage Location
Authentication Key (AK)	Proves the user/host is authorized to issue commands	Derived from user password, never stored in plaintext
Band Master Key (BMTK)	Protects individual locking ranges (bands) on the drive	Encrypted under the AK, stored on the NAND medium
Media Encryption Key (MEK)	The actual AES cipher key that encrypts/decrypts all user data on the NAND flash	Inside the controller's dedicated key memory — never leaves the controller die

1.4.2 What is MEK Rotation

MEK rotation is the act of replacing the current Media Encryption Key with a new one and securely discarding the old key. Within the TCG Opal framework, this is accomplished through the RevertSP and Revert methods, which are the Opal equivalents of the NVMe Sanitize (Crypto Scramble) command [**aronsson2025secure**].

The procedure, as implemented by Opal-compliant drives, works as follows:

1. The host issues a RevertSP (or Revert) command to the drive's TCG Storage Interface.
2. The controller generates a new random AES-128 or AES-256 key.
3. The controller overwrites its internal MEK register with the new key — this is a purely volatile operation within the controller logic.
4. The old MEK is destroyed as the register is overwritten. Since the MEK never leaves the controller in plaintext, and the controller has no mechanism to export it, the old key becomes unrecoverable.
5. All ciphertext on the NAND medium is now permanently undecryptable. The data has not been physically erased — every bit pattern remains exactly where it was — but it can never be recovered because the cryptographic secret that produced the plaintext is gone.

6. Optionally, per the NATO AEDP-3 standard, a block erase can follow the cryptographic erase to physically scrub the NAND cells, adding defense in depth [**aronsson2025secure**].

A full block erase on a large NVMe drive (e.g. 1 TB) can take tens of minutes because it must cycle through every NAND block's erase operation sequentially, with verification [**aronsson2025secure**] while MEK rotation completes in approximately 0.22 seconds regardless of drive capacity or how much data is stored

But still, these approaches has their practical limitations. Firstly, vendors can be inconsistent in supporting and implementing these commands, with some drives ignoring them or providing false assuarances of completion [**wang2020challenges**]. Further, Native commands are all-or-nothing where they sanitize the entire device which makes them suitable for decommissioning but useless in routine per-file deletions. And the controller's internal logic is not transparent, making it difficult to verify that the intended sanitization has actually occurred, all-zero reads may reflect the controller lying rather than physical erasure [**aronsson2025secure**]. Finally, wear-leveling mechanisms spreads data across cells, garbage collection creates copies in unexpected locations, and over-provisioning hides blocks from the host, all of which mean that even a correctly executed sanitize command may leave data remnants in retired or remapped blocks [**aronsson2025secure**]. A verifiable alternative method SAFE (Scramble and Finally Erase) was proposed to overcome these limitations. SAFE encrypts all data with a key held only in the controller's volatile memory, then on receiving a sanitize command: (1) erases the controller key memory, (2) erases every NAND block, (3) writes a known verification pattern, (4) erases again, and (5) reinitializes with a new key. This provides controller-level verifiability but, like the other native commands, offers no selective deletion capability [**reardon2013sok**].

SAFE (Scramble and Finally Erase) essentially performs an MEK rotation combined with a follow-up block erase and verification pattern — a verified MEK rotation plus physical scrub [**zhang2018ensuring**]. The Eraser and Holepunch schemes operate at a higher layer, using host-level master key rotation with tree-based or PPRF-based file key derivation to achieve the equivalent of selective "mini-MEK rotations" at file granularity, solving the all-or-nothing limitation of native NVMe MEK rotation [**aronsson2025secure**, **holepunch**].

Chapter 2

Comparative Analysis of Existing Literature

The growing dependence on digital systems for both personal and professional activities has resulted in an unprecedented accumulation of sensitive information. As organizations and individuals increasingly store confidential data on a wide range of digital media—ranging from traditional hard drives to solid-state drives and cloud environments—the need for reliable and verifiable data deletion mechanisms has become critical. Failure to ensure proper data sanitization can expose users to privacy breaches, corporate espionage, and regulatory non-compliance. From a cybersecurity perspective, secure deletion plays a dual role. On one hand, it is an essential component of data protection and privacy preservation, ensuring that information is irretrievably removed when no longer needed. On the other hand, it can also be employed as part of anti-forensic strategies, deliberately obstructing digital investigations by erasing potential evidence. Understanding this duality is fundamental to appreciating the technical and ethical challenges that surround secure deletion.

2.1 A Taxonomy of Anti-Forensic Techniques

The field of digital forensics is founded on the assumption that traces of user activity persist within digital systems and can be extracted for analysis and attribution. Anti-Forensics challenges this assumption by allowing the deliberate use of various techniques, procedures, tools and stratergies to obstruct the collection of digital evidences from a computer system. The main objective of these methodologies are to hinder Data Collection and impede Forensic investigations. These techniques aim to obscure, alter, or destroy digital evidence in ways that reduce the likelihood of recovery or interpretation. Anti-Forensics Techniques can be categorized into

four distinct categories based on their approach: **data hiding, artefact deletion, obfuscation of traces, and attacks on forensic tools and methods.** [**SystematicReview**]

Table 2.1: Summary of Anti-Forensic Techniques

Category	**What it does**	**Examples**
Data Hiding	Conceals data so it's difficult to find, while keeping it accessible to the adversary	Encryption, steganography, filesystem manipulation, Host Protected Area modification
Artifact Deletion	Permanently destroys or overwrites data to eliminate recoverable traces	Secure file deletion, disk wiping utilities, degaussing, physical destruction
Obfuscation of traces	Confuses or misdirects the forensic process	Log cleaners, timestamp alteration, spoofing, backbone hopping
Attacks Against Tools/Methods	Directly targets forensic software or procedures	Compression bombs, hash collision attacks, anti-reverse engineering

Secure deletion falls under the **artifact deletion** category, as it focuses on the permanent destruction of data to prevent recovery.

2.2 A Comparative Analysis of Secure Deletion Techniques

2.2.1 A Layer-Based Taxonomy of Secure Deletion

Secure deletion techniques can be organized into a layered taxonomy based on the layer of the storage stack at which they operate. Each layer offers a different interface to the physical medium, and the choice of layer involves a fundamental trade-off: lower layers give stronger irrecoverability guarantees, while higher layers make it easier to identify which data objects to delete [**reardon2013sok**]. Below

is the taxonomy, ordered from the lowest (physical) to the highest (application) layer, followed by cross-layer approaches that bridge multiple layers.

Physical Layer

The interface at this layer is the set of physical actions one can perform on the medium. Secure deletion here entails physical destruction: shredding, incineration, or use of an NSA/CSS-approved degausser. Magnetic media are securely deleted by degaussing; flash memory, optical discs, and punched cards are not — they require incineration or mechanical destruction. This layer achieves the strongest guarantee of irrecoverability but operates at full-device granularity only; no individual data object can be singled out.

Controller Layer

Controllers translate between the analog physical medium and the binary format used by higher layers. They expose standardized hardware interfaces such as SCSI or ATA, which offer reading and writing of fixed-size logical blocks [**reardon2013sok**]. Secure deletion at this layer takes these forms:

- **Full-device sanitization.** Both SCSI and ATA offer secure-erase or security-initialize commands that exhaustively overwrite every block. NIST recommends these as the non-destructive way to sanitize magnetic hard drives.
- **Block overwriting.** A single block can be overwritten to displace its old content, but without knowledge of which blocks constitute a particular data object, the controller cannot target individual files — it must sanitize every block to guarantee deletion of any one object.
- **Analog remnants** Gutmann demonstrated that on magnetic media, the precise analog voltage of an overwritten bit can reveal its prior value, motivating multi-pass overwriting schemes.
- **Flash controller caveats** Implementation of secure erase on SSDs is not always correct — some devices report success while the data remains accessible. Swanson et al. propose encrypting all data with a controller-stored key, then erasing that key before performing a three-pass sanitization.

Block Device / Device-Driver Layer

Device drivers are software abstractions that consolidate hardware access behind a common interface. Two distinct interfaces are relevant:

- **Block device driver** : Permits reading and writing of logically addressed blocks; used for magnetic hard drives and SSDs fronted by an FTL.
- **Memory Technology Device (MTD)** : Accesses raw flash memory directly. MTD permits reading and writing, but blocks must be erased before being written, and erasure occurs at a much coarser granularity than reads and writes. Unsorted Block Images (UBI) builds on MTD to simplify raw-flash access.

At this layer, the device driver cannot distinguish between valid data and deleted data without guidance from higher layers — a limitation that motivates the cross-layer approaches described in latter section.

File System Layer: In-Place Updates

For physical media whose device driver supports in-place updates (primarily magnetic hard drives), secure deletion can be integrated directly into the file system:

- **ext2 secure-deletion patch** : Adds a sensitive attribute for files. When a sensitive file is deleted, the freed blocks are passed to a kernel daemon that overwrites them asynchronously, sorted by disk seek position for efficiency.
- **ext3 secure deletion** : Two variants: a basic mode that overwrites file data once, and a comprehensive mode that overwrites both data and metadata using a configurable scheme such as Gutmann's 35-pass pattern. Both can be restricted to files with a sensitive extended attribute.
- **Secure deletion via renaming** : Intercepts unlink and truncate events, moving affected files into a special secure-deletion directory where a background process (e.g., shred) sanitizes them.
- **Purgefs** : A file-system extension for any block-based file system that overwrites blocks when they are returned to the free list, supporting per-file sensitivity flags.
- **Versioning file systems** : Uses all-or-nothing cryptographic transforms: each data block is expanded into an encrypted block plus a key-sized tag. Deleting a file version requires overwriting only the version's tags, leaving the shared encrypted data block intact but undecipherable.

File System Layer: No In-Place Updates

For flash memory and other media that do not permit in-place updates, overwriting is not a viable strategy. These approaches instead work within the constraint that data can only be destroyed by erasing an entire erase unit:

- **Compaction (immediate)** : Copy all valid data out of the erase unit containing the deleted data, then erase the unit. This is costly: one erasure per deletion, plus the read-copy overhead.

- **Batched compaction** : Accumulate deletions and compact all affected erase units intermittently, amortizing the erasure cost. Log-structured file systems already perform a similar technique (garbage collection), but prioritize space reclamation over deletion latency.

- **Per-file encryption** : Encrypts each file with a unique key stored in the file header. By colocating all header versions on the same erase unit, deleting a file requires compacting only one erase unit — the one holding the key — under computational assumptions.

- **DNEFS** : Encrypts every data block with a unique key and colocates keys in a dedicated key-storage area. Secure deletion is performed intermittently by replacing the key-storage area with a new version, omitting keys for deleted blocks. This achieves many-block deletion with few erasures.

- **Scrubbing** : Reprograms all 1 bits on a programmed page to 0 without erasing the block. Scrubbing destroys data without requiring compaction, but carries a reliability cost: cell disturbance shifts the threshold voltage of adjacent cells, increasing bit error rates. In 3-D NAND, as many as four adjacent pages can be disturbed[**3dnandwang2018**].

- **Partial scrubbing and down-bit programming** Partial scrubbing modifies a subset of bits to avoid the tell-tale all-zero pattern. Down-bit programming rewrites a TLC page in SLC mode, compacting the data into a single-bit representation and destroying the original multi-bit content[**ahn2019schemes**].

Application Layer

User-level approaches interact only through a POSIX-compliant file-system interface. They are easy to deploy but have the least control over the physical medium:

- **Secure Erase utility (UCSD)** : A user-level application that invokes the hardware controller's secure-erase command, propagating a lower-layer capability to the user interface. Erases the entire device.

- **File-overwriting tools (srm, wipe, shred)**: Open the file from user space, overwrite its contents multiple times with insensitive data (zeros, random patterns), then unlink. These tools fail silently on file systems that do not perform in-place updates, such as journaling or log-structured file systems.

- **Free-space filling tools** Fill all unused blocks of the file system, thereby overwriting any deleted data that still resides in free blocks. Cost is proportional to free space; on flash media, the high wear from filling can be mitigated by perpetually maintaining free space within a target range.
- **Database secure deletion** MySQL and SQLite offer compile-time or run-time options to overwrite deleted records with zeros. However, this relies on the underlying file system to ensure that overwritten data is actually replaced at the physical level — an assumption that does not always hold.
- **Revocable backup system** Encrypts each backup with a unique key, then re-encrypts keys under a temporal master key hierarchy. Backups are deleted by not re-encrypting the corresponding key in the next time interval — the first cryptographic secure-deletion scheme.

Cross-Layer Approaches

These approaches, as described in Chapter 1, bridge the file system and device-driver layers by passing information about deleted blocks downward, enabling efficient sanitization at layers that otherwise lack the semantics to identify deleted data:

- **TRIM (ATA DATA SET MANAGEMENT)** : The file system notifies the SSD controller about logical blocks that no longer store valid data. Originally designed as a performance optimization (to prevent flash thrashing), TRIM gives the controller knowledge of which blocks are discardable. However, TRIM is asynchronous — the OS receives no confirmation of physical erasure — and it provides no mechanism to restrict notification to sensitive blocks only.
- **TrueErase**: Adds a dedicated communication channel between the file system and the device driver. Unlike TRIM, TrueErase only notifies the driver about blocks belonging to files explicitly marked as sensitive, and it triggers immediate sanitization rather than deferring to the controller's scheduler.
- **ZK TRIM (Zombie-Kill)**: Extends TRIM with three modes — normal (defer to garbage collection), immediate (trigger compulsory GC on the TRIM'd blocks), and suspension (delay compulsory GC by a configurable threshold, then enforce it). Immediate mode limits the zombie time but incurs up to 68x higher I/O response latency.

Table 2.2: Secure Deletion Approaches by System Layer

Layer	**Approaches**	**Strengths**	**Limitations**
Physical	Shredding, incineration, degaussing	Unconditional irrecoverability	Full-device only; destroys medium
Controller	ATA/SCSI Secure Erase, block overwriting, multi-pass (Gutmann)	Well-defined hardware interface; works on any block device	No file-level granularity; FTL remapping may foil overwrites
Driver / Block Device	Block device driver, MTD, UBI	Software abstraction; MTD exposes raw flash	No visibility into which blocks hold deleted data
FS (In-Place)	ext2/3 secure deletion, Purgefs, versioning FS crypto (Peterson)	File-aware; balances granularity and control	Requires in-place-update device driver; journaling may preserve data
FS (No In-Place)	Compaction, batched compaction, DNEFS, scrubbing, per-file encryption (Lee)	Works under erase-before-write constraint; encryption amortizes erasures	High cost for compaction; scrubbing causes cell disturbance
User / Application	Secure Erase utility, srm/wipe/shred, free-space filling, DB secure deletion	Easy to deploy; no kernel modification	Silently fails on log-structured / journaling FS; least physical control
Cross-Layer	TRIM, TrueErase, ZK TRIM (Zombie-Kill)	Bridges FS semantics with driver-level sanitization; per-file sensitivity possible	TRIM is asynchronous & unverifiable; ZK TRIM imposes I/O latency

2.2.2 Comparative Evaluation of Secure Deletion Techniques

To compare methods across different layers, a consistent set of evaluation dimensions is required. The following dimensions are derived from the literature and from the technical constraints established in Chapter 1. Each dimension captures a distinct trade-off that system designers and compliance officers must consider.

Layer of Implementation

The layer at which a secure deletion technique operates fundamentally shapes its capabilities, each abstraction layer between the user and the physical medium complicates secure deletion in a different way: the controller may remap LBAs to hide bad blocks, the FTL performs out-of-place updates that preserve old versions, the file system may journal writes that retain stale copies, and the OS cache may hold sensitive data in memory long after the file is unlinked. The choice of layer embodies a fundamental trade-off — lower layers provide the strongest irrecoverability guarantee because no abstraction stands between the deletion action and the storage cells, but they operate at full-device granularity only and cannot identify individual files. Higher layers can pinpoint exactly which data object to remove, but they have the least control over whether the deletion actually reaches the physical medium, making them vulnerable to FTL remapping, journaling, and copy-on-write semantics that silently preserve old data. Understanding the layer at which a given approach operates is therefore the first and most critical step in evaluating whether it can meet a given security requirement.

Deletion Latency

The deletion latency refers to the time from deletion command issuance to data being unrecoverable. Approaches divide broadly into immediate and delayed categories: an immediate approach offers a negligibly small latency, while a delayed approach defers physical erasure to a later garbage collection or compaction cycle. On flash-based storage, this distinction is most visible in the zombie time — the interval between when the FTL marks address mappings as invalid and when garbage collection physically erases the NAND cells. During the zombie time, the data remains fully intact and recoverable through low-level access, even though the file system and the OS consider it deleted.4 The duration of the zombie time is highly workload-dependent: under read-heavy workloads, the free-block pool remains ample and garbage collection may never activate the blocks containing deleted pages, making the zombie time effectively unbounded. Latency is therefore not a fixed property of a deletion mechanism but emerges from the interaction between the mechanism and the device's internal state and workload pattern.

Write Amplification Factor

Write Amplification Factor (WAF) represents the additional writes caused by deletion operations that also reduces the SSD lifespan. NAND flash has a limited program-erase (P/E) cycle budget per block — typically 1,000-10,000 cycles for TLC — and every deletion operation that triggers an erase consumes part of that budget. Three FTL-internal processes compound this: garbage collection copies valid pages from a victim block to a free block before erasing, wear leveling migrates cold data to balance P/E cycles across blocks, and bad-block management relocates data away from failing regions — each migration creates an extra physical write that must be separately sanitized for deletion to be complete. The write amplification factor (WAF) captures this overhead as the ratio of physical writes to logical writes; a WAF of 3 means every logical write causes 3 physical writes on the NAND medium. Aggressive secure deletion policies such as immediate-mode ZK TRIM can increase I/O response latency by 68x on average by forcing compulsory garbage collection on every deletion, though suspension-mode ZK TRIM reduces this penalty by batching deletions within a configurable time window. For scrubbing-based approaches, the disturbance to adjacent cells from each reprogramming pass increases the bit error rate, potentially triggering further read-copy-erase cycles that amplify wear still more.

Crash Consistency

Whether deletion guarantees survive power loss or system crash. In cryptographic secure-deletion schemes, the key state (master key, key-encryption-key hierarchy, or PPRF state) and the on-disk data it protects must remain in sync at all times: if a crash occurs after a key is rotated but before the on-disk key table is updated, the file system is left with encrypted data whose decryption key is permanently lost or, worse, with stale keys that still grant access to supposedly deleted files. ERASER, the prior state-of-the-art, suffers from precisely this problem — after a crash during master-key rotation, laboratory experiments show that up to 64 files may become irrevocably lost or recoverable due to desynchronization between the TPM-hosted key store and the on-disk file-key table. Holepunch resolves this by embedding a journaling scheme in the key-management path: before any puncture or rotation operation, the intended change is written to a dedicated journal region on disk, and if a crash interrupts the operation, the journal is replayed on recovery to bring the key state back to a consistent point. For physical-erasure approaches, crash consistency is less fragile — an interrupted erase leaves the NAND cells in a partially erased state that is at least as secure as before the command — but the metadata describing which blocks were meant to be erased may be lost on power cycle, meaning the system cannot confirm post-crash whether the deletion was completed.

Granularity of Deletion

The size of the smallest data unit that a secure deletion approach can independently target. The SoK taxonomy divides granularity into three categories: per-physical-medium (sanitizes the entire device — useful only for decommissioning, not routine operation), per-file (targets individual files as the deletion unit), and per-data-block (sanitizes at the smallest unit the physical medium offers, such as a sector or NAND page).1 Per-file granularity is the most intuitive for users, but it can be deceptive: long-lived files such as databases or virtual machine images store many distinct data objects within a single file, so deleting the file means deleting everything inside it — an extraordinary measure that is impractical as a routine defense against a disclosure-timing adversary. Conversely, per-data-block granularity provides fine-grained control but introduces its own complications: scrubbing disturbs adjacent cells, the sanitized all-zero pattern is itself detectable by forensic analysis, and approaches that work at the block or erase-unit level must migrate valid colocated data before erasing, incurring write amplification proportional to the valid-data density of the victim block. A related dimension is scope — whether an approach is untargeted (sanitizes all deleted data, guaranteeing no false negatives) or targeted (only sanitizes files explicitly marked as sensitive). Targeted approaches are more efficient but introduce usability risk: users must correctly classify every sensitive file, and a single false negative can leak confidential data. The choice of granularity therefore has cascading effects on performance, wear, forensic detectability, and user error tolerance.

Verifiability Level

Ability to independently confirm that deletion actually occurred. Most deployed schemes provide no verification at all — the host simply trusts the controller's completion signal (e.g., the return value of an ATA Secure Erase command), even though studies have found controllers that report success while leaving the data fully intact. Cryptographic erase is inherently unverifiable at the physical level: discarding a key makes data computationally inaccessible but does not alter the stored ciphertext, so a laboratory adversary with physical access can still recover the encrypted bits and may decrypt them in the future if the encryption is broken or the key is later reconstructed. Ahn and Lee proposed an in-band verification technique for NAND flash that exploits the threshold-voltage distribution of sanitized cells: under normal program conditions, the number of cells in each Vt state is approximately uniform, but after a successful block erase or scrubbing, the distribution shifts to a characteristic non-uniform pattern. A simple read operation can count cells per Vt distribution and compare the result against a manufacturer-set threshold, determining pass or fail without exposing the original personal data. This check requires no changes to existing NAND flash hardware and can be

performed immediately after any deletion operation.17 A higher-assurance variant uses an XOR between known data and the sensed residual from the sanitized area, passing only if the residual falls within the ECC-correctable range — confirming physical destruction to within a small, quantifiable error margin.

Regulatory Alignment

Whether the method meets NIST Purge level or GDPR Article 17 requirements. NIST SP 800-88 Rev. 1 and the newer IEEE 2883-2022 define three sanitization tiers: Clear, Purge and Destroy. Overwriting-based methods on flash are currently categorized as Clear rather than Purge because they are not standardized or vendor-validated for that purpose, while erasure-based methods and cryptographic erase qualify as Purge when implemented correctly.20 GDPR Article 17 does not prescribe specific technical mechanisms but requires that data be rendered irrecoverable across all storage locations, including backups and caches. In practice, this means deletion mechanisms must survive device decommissioning, outsourced cloud storage, and unexpected data duplication by the storage stack — a bar that standard file deletion or TRIM alone cannot meet because neither physically destroys the data nor confirms its destruction. The practical implication is that regulatory compliance demands multi-layer strategies: cryptographic erase for at-rest data, verifiable physical erasure for decommissioned devices, and auditable deletion logs to demonstrate compliance to regulators.

2.2.3 Comparison Matrix

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches for Flash Storage

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Overwrite (user-level, shred/dd)	User / OS	File	Seconds to minutes (depends on file size)	High: Full file rewrite + FTL out-of-place update creates additional physical writes	None: No guarantees; power loss during overwrite leaves partial data	Host cannot confirm physical overwrite occurred; FTL may have mapped writes elsewhere	Does not meet NIST Clear, Purge, or Destroy for SSDs

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Scrubbing (page overwrite)	Device / FTL	Page	390 μs per page	High:Cell disturbance forces valid page migration from adjacent pages; each scrub consumes part of scrub budget; increased BER triggers ECC correction overhead	None: Power loss during scrubbing leaves page in indeterminate Vt state; adjacent cells may be corrupted	Partial: Vt distribution analysis can distinguish scrubbed cells from native zeros ; requires special equipment	No: Not approved by NIST SP 800-88 or IEEE 2883-2022

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Analog sanitization	Device / FTL	Page	390 μs to 1 ms per page	Low: Reprograms to Vt distribution indistinguishable from native zero; less cell disturbance than scrubbing	None: Power loss during multi-step Vt programming leaves intermediate state	High: Vt distribution statistically indistinguishable from native 0-programmed page; forensic recovery infeasible	No: Not standard-approved; research only
Down-bit programming	Device / FTL	Page (MLC/TLC)	<500 μs per page (single operation)	Very low: TLC→SLC conversion requires one program operation; no erase needed	None: Power loss during mode conversion leaves page in undefined state	Low: Residual Vt distribution may still reveal original data pattern; forensic detection possible	No: Not standard-approved; research only

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
TRIM (standard)	OS → FTL	LBA range (block)	Unbounded (seconds to indefinite – zombie time)	None : TRIM itself adds no writes; GC adds WAF 1.1–3× depending on workload	None: TRIM commands are asynchronous; power loss erases pending TRIM queue; data remains on NAND	None: Host receives no confirmation; controller may ignore TRIM entirely; no way to verify physical erasure	No: Clear level only (NIST SP 800-88); does not guarantee physical removal
ZK-TRIM Immediate	FTL	Block	Bounded by GC execution time (I/O penalty: 68× normal operation)	High: Compulsory garbage collection on every TRIM; forces valid page migration even when free blocks available	None: Power loss during compulsory GC leaves victim block in partially erased state	None: Same as standard TRIM – no confirmation of physical erasure	No: Research only; not deployed in commercial drives

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
ZK-TRIM Suspension	FTL	Block	Bounded	Medium: 60× I/O penalty compared to normal operation; fewer GC events than Immediate mode	None: Power loss during suspension window or GC leaves data recoverable	None: Same as standard TRIM – no confirmation	No: Research only; not deployed
Secure Erase (ATA/ eMMC)	Controller	Block / Device	2.5–8 seconds (device-level)	Medium: Full block erase of every block; no valid page migration needed; one erase cycle per block	None: Power loss during erase leaves some blocks erased, others untouched – inconsistent state	Low: Vendor-dependent; some drives return all-zero reads without physical erase (controller masking) ; no independent verification	Partial: Clear level only (NIST SP 800-88); not Purge

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	Layer	Granularity	Deletion Latency	WAF	Crash Consistency	Verifiability	Regulatory Alignment
NVMe Sanitize Block Erase	Controller	Device	5–8 seconds (small drive) to tens of minutes (1 TB+)	Medium: One erase cycle per block; no valid page migration needed	None: Power loss during erase leaves device in inconsistent state; NVMe spec defines no recovery semantics	Low: Controller may report success without physical erase (masking); returns all-zero reads as abstraction layer lie	Yes: Purge level (NIST SP 800-88, IEEE 2883-2022)

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
NVMe Sanitize Crypto Erase (key discard)	Controller	Device	0.22 seconds (capacity-independent)	None: No NAND write operations; only key register overwritten	Vulnerable: Power loss during key register write leaves old key partially overwritten; undefined behavior; no recovery spec	None: Host receives completion status but cannot confirm key was destroyed; controller may preserve key in debug area or retired key table	Yes: Purge level (NIST SP 800-88, IEEE 2883-2022)

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
NVMe Sanitize Crypto Erase (full re-encryption)	Controller	Device	Minutes to hours (capacity-dependent)	High: Rewrites every user block (decrypt with old MEK, encrypt with new MEK, write back)	Vulnerable: Power loss during re-encryption leaves some blocks with old key, some with new key – data may be partially unrecoverable or corrupt	Low: Host sees completion but cannot verify each block was re-encrypted; controller may skip blocks	Yes: Purge level (vendor-dependent implementation)

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Cryptographic Erasure (per-block key)	OS / FS	Block / File	Milliseconds (key deletion only)	None: Key deletion only; no NAND writes	None: Power loss during key table update leaves key references inconsistent	None: No mechanism to confirm key was destroyed; key may remain in memory or swap	Yes: Purge level if key destruction is verifiable (assumption only)

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Eraser (tree-based CE)	Block device / Kernel	File	<1 second (branch revocation)	None: Key operations only; no additional NAND writes beyond metadata	Weak: TPM desync on crash; post-crash experiments show up to 64 files lost when master key rotation desynchronizes from on-disk file key table	None: TPM black box; host cannot confirm master key was revoked; no audit trail	Partial: Meets Purge cryptographically but crash vulnerability creates compliance risk under GDPR Article 17

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
Holepunch (PPRF-based CE)	Block device / Kernel	File	0.22 seconds (puncture operation)	None: No per-file key storage; puncture modifies only master PPRF key; no additional writes beyond journal	Strong: Journaled puncture scheme; crash-consistent – on recovery, journal replays pending punctures; no file loss or undeleted files	None: Same as Eraser – host cannot verify puncture occurred; TPM black box	Yes: Purge level + crash safe; strongest regulatory alignment among file-level CE

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
TrueErase (cross-layer)	FS + Driver	File	Fast (immediate at driver notification)	Low: Selective notification; only sensitive files trigger immediate actions; no blanket TRIM	None: Power loss during driver notification leaves file in indeterminate state	None: No verification mechanism proposed	Partial: No standard approval; cross-layer approach not recognized by NIST
SAFE (controller-level)	Controller	Device	Slow (multi-step: key erase + block erase + pattern write + re-erase)	High: Triple erase	Limited: Power loss during multi-step sequence leaves device inconsistent; verification pattern may be partially written	Medium: Verification pattern written and checked; controller can attest completion; host can read pattern to confirm	Yes: Purge level + verifiable; but device-only

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
RedFlash (OOB duplicate chain)	FTL / Controller	Page / Block	Medium (chain traversal + erase; no specific benchmark)	Medium: Additional OOB writes to maintain duplicate chain; each write adds metadata	None: Power loss during chain update leaves duplicates untracked; OOB metadata may be corrupted	None: No verification mechanism; depends on correct chain traversal	No: Research only; not deployed
Secure Copy-Back	FTL / Controller	Page	Low (single program operation)	Low: No extra erase; programs source page with random data or deletion pulses	None: Power loss during source page programming leaves partial destruction	None: No verification that source page was actually sanitized	No: Research only; not deployed

Continued on next page

Table 2.3: Comprehensive Comparison of Secure Deletion Approaches (continued)

Approach	**Layer**	**Granularity**	**Deletion Latency**	**WAF**	**Crash Consistency**	**Verifiability**	**Regulatory Alignment**
SAS Deletion (scrub-aware)	FTL / Controller	Page / Block	Optimized: 82% reduction in scrubbing latency (70 µs per page)	Controlled: Scrub budget limits total scrubs per block; valid page migration when budget exceeded	None: Same as scrubbing – power loss leaves indeterminate state	Medium: Cell-count verification possible; Vt distribution analysis	No: Research only; not standard-approved
Physical destruction	Physical	Device	Permanent (seconds to minutes, irreversible)	N/A: Device destroyed; no further writes possible	N/A: Device non-operational after destruction	High: Physical inspection can confirm destruction (e.g., crushed, shredded, incinerated)	Yes: Destroy level (NIST SP 800-88, IEEE 2883-2022)

2.3 Synthesized Observations from the Literature

The secure deletion literature spans a wide range of approaches and evaluation methods, reading across the literature reveals patterns and several persistent challenges in secure deletion techniques, particularly in modern storage environments.

Layer Seperation Problem :

Secure deletion in a modern storage system is fundamentally complicated by the layered architecture through which data passes. Each layer - application, file system, block device, controller (FTL), and physical medium – provides its own interface to the layer above and introduces its own abstraction over the layer below. At the physical layer one can ensure true irrecoverability, but at the user layer one can easily identify which data object to delete. The two capabilities never coincide at the same layer. The FTL abstracts NAND flash behind a standard block-device interface so that traditional file systems remain compatible, but this same abstraction hides the physical state of data from the host. When raw flash is accessed directly, a different controller interface is exposed; when accessed through an FTL, overwriting a logical block does not replace its physical content – it creates two versions, one of which persists.

Secure deletion implemented at any single layer is systematically incomplete. Performing secure deletion in one layer cannot eliminate data because leakage may persist in the other layers. There is no one ideal layer for secure deletion because the criteria are in tension. Low-layer approaches (controller-level block erasure) require fewer assumptions about the interface's behavior and provide physical irrecoverability, but cannot select individual data objects for deletion. High-layer approaches (file-system-level or cryptographic erasure) let users clearly mark which data to delete, but must trust that the layers beneath faithfully execute the instruction. This is not an implementation bug; it is a structural property of layered storage architectures, and no single-layer solution can close it.

Verifiability Crisis :

After performing a secure deletion operation, there is currently no reliable, independent, and non-destructive way to confirm that the data has been physically destroyed. This is the verifiability crisis, and it cuts across every approach category. Cryptographic erasure is inherently opaque. When a key is deleted from a TPM or key store, the storage device may return zeroed data at the abstraction layer, but this does not confirm that the underlying physical cells have been sanitized or even that the key has actually been removed. TPMs are themselves black boxes with limited externally accessible mechanisms to confirm key deletion independently. Because the binding between key revocation and physical inaccessibility is mediated entirely by the encryption scheme, verification collapses to trusting that the key was truly erased, no copies of the key exist elsewhere, and no side-channel or future cryptanalytic advance can recover the plaintext. None of these are independently

verifiable after the fact.

Physical erasure has the opposite profile. A block-erase command physically removes charge from NAND cells. In principle this is verifiable through electrical inspection of Vt distributions, but practical verification at scale remains expensive, destructive, and vendor-dependent. Scrubbing provides page-level erasure but introduces disturbance effects that complicate assessment of whether complete data removal was achieved. The irony is that the approaches with the strongest physical guarantees have the weakest practical verification paths.

Standards such as NIST SP 800-88 Rev. 1 and IEEE 2883-2022 define sanitization levels - Clear, Purge, and Destroy - that require "demonstrable and verifiable data removal evidence" at each level. Yet no deployed technique today can independently demonstrate that a Purge-level operation on an SSD has succeeded without destructive physical inspection. The crisis is not merely technical: regulatory mandates for demonstrable deletion exist, but the technical means to satisfy them do not, creating a gap between legal obligation and practical verification.

Performance-latency Trade-offs :

Secure deletion oscillates between two poles: immediate guarantees with severe performance cost, or deferred guarantees with unbounded latency. This tension is structurally embedded in how flash storage manages physical erasure. After the FTL invalidates a file's address mapping, the physical data on the NAND cells remains intact until garbage collection erases the containing block. The zombie time is the core latency parameter of any erasure-based scheme. Its duration is not fixed: it depends on write pressure, over-provisioning ratios, FTL victim selection policy, and whether TRIM was issued.

The trade-off extends beyond GC to the approach category itself. Delayed approaches reduce wear and I/O overhead but introduce a fixed worst-case upper bound on deletion latency that may be minutes, hours, or longer. Purging and ballooning techniques address this by deliberately filling the device with junk data to force GC, but the erasing time scales with device capacity and write speed. Performance overhead is not limited to erasure-based methods, even scrubbing incurs a performance decrease due to overhead. Every performance-enhancing mechanism inside SSDs is counterproductive to secure deletion guarantees.The pattern is clear: at one end, cryptographic erasure achieves sub-second deletion latency regardless of device usage but provides no physical guarantee. At the other end, immediate-mode compulsory GC provides bounded physical erasure but at a 68x I/O penalty. There is currently no technique that achieves both bounded physical erasure and negligible performance cost on commodity hardware.

Shifting Trust, Not Eliminating It :

Cryptographic erasure is often presented as a clean solution, delete the key, and the encrypted data becomes irrecoverable. The literature reveals that this framing obscures multiple layers of trust that cryptographic erasure merely relocates rather

than removes. Every secure deletion approach relies on a trust model, an assumption about which components behave correctly and can be observed. Cryptographic erasure replaces trust in the physical storage medium with trust in a cryptographic key store. This is a favorable trade for most deployment contexts, but it is a shift of trust, not its removal. The approach that is strong on security guarantees is simultaneously weak on verifiability. It assumes that encrypted data, once the key is destroyed, is computationally irrecoverable. This is a forward-looking assumption that depends on the encryption algorithm remaining secure against future cryptanalytic advances. For long-term confidentiality requirements, this is a significant assumption that physical erasure does not require. FTL operations create duplicate copies of data at the physical level, the same problem applies at the key management level: if duplicate copies of encryption keys exist in journal files, crash recovery logs, or backup metadata, cryptographic erasure fails silently because the attacker can recover the key from a copy the OS does not know exists. Verifying that all key copies have been eliminated requires the same kind of system-wide visibility that physical erasure struggles with.

Absence of Crash Consistency in Regulatory Frameworks : The patterns across above discussed systems reveals a structural principle: the crash consistency of a secure deletion system is determined by whether key metadata and data metadata share the same durability domain. When key state is maintained in a separate trusted component (TPM) while data metadata is on disk, the interface between them becomes the consistency bottleneck. Any system that crosses durability domains must explicitly address this interface or accept the crash consistency gap. The GDPR right to erasure (Article 17), the CCPA's right to deletion, and standards including NIST SP 800-88 Rev. 1 and IEEE 2883-2022 mandate timely, demonstrable data removal at defined sanitization levels. None of these frameworks describe any requirement for crash-consistent deletion. They specify what must be deleted and at what assurance level, but they are silent on what happens when the deletion process itself is interrupted by a power failure or system fault. A system could be fully compliant with every regulatory sanitization mandate and still leave data in an inconsistent, partially recoverable state after a crash during deletion.

Large cloud providers describe deletion as an iterative process requiring up to 180 days due to data replication and backup cycles, far exceeding the timelines implied by GDPR Article 17. At that scale, crash events during the deletion window are a statistical certainty rather than a corner case. Regulatory standards that specify a deletion outcome but not the reliability of the deletion process create a dangerous gap: a provider can certify compliance based on policy while the technical mechanism silently fails under crash conditions. Shastri et al.'s identification of six GDPR anti-patterns in cloud-scale systems – storing data without clear deletion timelines, reusing data indiscriminately, and risk-agnostic data processing – reinforces that achieving regulatory compliance requires systemic

architectural changes, not point solutions. Crash consistency is a prime example of a systemic architectural requirement that regulation has not yet addressed.

2.4 Empirical Evaluation Approaches in the Literature

Experimental evaluations in the literature vary widely in methodology. Reardon et al. conducted controlled tests examining block-unaligned truncations and sparse files across multiple file systems to evaluate practical secure deletion approaches [**reardon2013sok**]. Aronsson and Lund performed timing measurements of block erase and cryptographic erase on commercial SSDs (SN7100 and SN850X), revealing vendor-specific inconsistencies and the need for device-independent verification methods [**aronsson2025secure**]. Wang et al. compared encryption-based, overwriting-based, and erasure-based methods across storage technologies, categorizing trade-offs in security guarantees, performance overhead, and practical feasibility [**wang2020challenges**]. The scrubbing-aware design by Wang et al. (2018) used a flash-memory simulator to evaluate deletion efficiency, achieving 82% average response time reduction through optimized allocation of sensitive data [**3dnandwang2018**].

The key methodological gap across these studies is the absence of a standardized, reproducible evaluation framework. Experiments are conducted on specific hardware with vendor-specific firmware behavior, making cross-study comparisons unreliable. There is no agreed set of metrics for measuring deletion effectiveness, performance impact, or verifiability.

2.5 Legal and Ethical Dimensions

2.5.1 GDPR and the Right to Erasure

The General Data Protection Regulation (GDPR), introduces the right to erasure (Article 17) as a foundational data protection obligation that directly intersects with secure deletion. Article 17(1) requires the controller to erase personal data without undue delay where, among other grounds, the data are no longer necessary for the purposes for which they were collected, consent has been withdrawn and no other legal ground applies, the data have been unlawfully processed, or erasure is required by Union or Member State law. The right is not absolute and the Article 17(3) provides exceptions where continued processing is necessary for freedom of expression and information, compliance with a legal obligation, the performance of a task in the public interest, public-health reasons, archiving or research purposes, or the establishment, exercise, or defence of legal claims. Recital 65 explains that

the right to erasure is particularly relevant where continued retention infringes the GDPR or other applicable law. It also confirms that further retention may remain lawful where it is necessary to protect competing interests, including freedom of expression and information, legal obligations, public-interest tasks, research, archiving, or legal claims. Recital 66 extends the right to the online environment. Where a controller has made personal data public and is obliged to erase them, it must take reasonable steps, including technical measures and measures directed towards other controllers, to inform them of the request to erase links, copies, or replications of those data. Article 5(2) is the point at which the legal framework becomes directly relevant to secure deletion. It requires the controller to be responsible for, and able to demonstrate compliance with, the GDPR principles. This accountability obligation must be read together with the storage-limitation principle in Article 5(1)(e) and the requirement in Article 32 to implement technical and organisational measures appropriate to the risks of processing [**gdpr2016**]. The GDPR therefore establishes an obligation to erase personal data when the legal conditions are satisfied, but it does not define a single technical meaning of "erasure." It does not require every controller to use overwriting, cryptographic erasure, block erasure, or physical destruction in every case. The appropriate technique depends on the storage medium, the sensitivity of the data, the possibility of replication, the intended reuse or disposal of the equipment, and the risks associated with recovery.

Politou et al. (2018) provide a comprehensive analysis of these provisions, noting that the right encompasses both the erasure of personal data and the broader right to be forgotten, requiring deletion from all possible sources in which the data resides, including third-party controllers to whom the data has been disseminated [**Politou2018**]. The GDPR introduces the withdrawal of consent as a trigger for erasure, a condition not present in prior data protection frameworks, and obligates controllers to take reasonable steps, including technical measures, to inform other controllers processing the data of the erasure request. Politou et al. argue that enforcement is feasible provided that implementation guidelines and low-level business specifications are put in place in a clear, cross-platform manner [**Politou2018**]. Shastri et al. (2019) identify six GDPR anti-patterns arising from the design and operation of modern cloud-scale systems: storing data without a clear timeline for deletion, reusing data indiscriminately, creating walled gardens and black markets, risk-agnostic data processing, hiding data breaches, and making unexplainable decisions [**Shastri2019gdpr**]. They highlight that Articles 5 and 13 of the GDPR require controllers to specify retention periods at the point of collection and to ensure personal data is deleted when no longer necessary for the purpose for which it was collected. In practice, large cloud providers describe deletion as an iterative process that can take up to 180 days due to data replication and backup cycles far exceeding the timeline implied by the regulation.

The GDPR does not, however, define what constitutes full deletion or erasure

at a technical level, leaving a significant gap between legal requirements and technical implementation [**AlgorithmicDestruction**]. The regulation requires that controllers take reasonable steps to inform other controllers of erasure requests [**vanOoijen2019**], creating an obligation without specifying the means by which compliance is to be achieved. Legal analysis of the landmark Google Spain case further illustrates the territorial and procedural complexity of deletion obligations, where search engine operators were held responsible as data controllers for processing personal data linked through search results [**AlgorithmicDestruction**].

2.5.2 Disposal and reuse of storage media

The lifecycle of storage equipment creates a separate and concrete deletion risk. A device may no longer be used by the original controller while still containing recoverable personal data in user-accessible space, remapped blocks, reserved areas, temporary files, or other parts of the storage architecture. Disposal and reuse must therefore be treated as data-protection operations rather than as ordinary asset-management activities. The Italian Garante addressed this issue in its provision of 13 October 2008 concerning electrical and electronic waste and the security of personal data. The Garante required organisations that dispose of, reuse, or recycle equipment containing personal data to adopt suitable measures to prevent unauthorised access. For reuse or recycling, the equipment must contain no personal data that remain present or intelligible. The measures may include secure erasure, making the data unintelligible, or physical destruction of the relevant media, taking account of the state of the art and the sensitivity of the information [**garante2008_electronic_waste**]. The Garante's accompanying practical guidance also emphasises that ordinary operating-system deletion is not sufficient for secure disposal. It describes secure erasure, overwriting, degaussing where appropriate, and physical destruction as possible measures, depending on the type of medium and the required level of protection [**garante2008_secure_deletion**]. ISO/IEC 27040:2024 provides a more current technical framework for storage security. It covers the protection of storage devices and media, related management activities, applications and services, and the period during which equipment remains in use and after its end of use or end of life [**iso27040_2024**]. The standard is therefore relevant to the selection and documentation of deletion methods across different storage technologies, including storage devices whose internal behaviour is not fully visible to the host system. ISO/IEC 27001:2022 Annex A control 7.14 addresses the secure disposal or reuse of equipment. Its requirement is that equipment containing storage media be verified to ensure that sensitive data and licensed software have been removed or securely overwritten before disposal or reuse [**iso27001_2022**]. The control is significant because it expressly includes verification. A deletion command or a user-interface message indicating completion should

not automatically be treated as sufficient evidence that the requirement has been met. The relationship between ISO/IEC 27001 and ISO/IEC 27002 should also be stated accurately. ISO/IEC 27001 defines requirements for an information-security management system, whereas ISO/IEC 27002 provides guidance on information-security controls, including the control corresponding to secure disposal or reuse [**iso27002_2022**]. Compliance with an ISO standard does not, by itself, establish compliance with the GDPR. The standards provide a risk-management and control framework that may help a controller demonstrate appropriate technical and organisational measures.The importance of this guidance for modern flash storage is not that its older overwriting examples remain universally appropriate. Rather, it demonstrates that disposal requires a deliberate and technically suitable sanitisation decision.

2.5.3 Accountability and evidencing deletion

The technical results of this thesis become legally significant through Article 5(2). If no deletion method can always be verified independently and non-destructively, the controller must determine what evidence can reasonably demonstrate that the deletion obligation was fulfilled. Article 5(2) should not be interpreted as requiring impossible proof of the physical state of every storage cell. It requires the controller to be able to demonstrate compliance. The relevant question is therefore not whether the controller can establish absolute metaphysical certainty that no residual bit exists anywhere, but whether the controller selected and implemented a deletion process that was appropriate to the risk and supported that process with credible evidence [**gdpr2016**].

A defensible deletion record should establish at least the following:

1. The controller identified the personal data covered by the erasure obligation and determined where those data, replicas, backups, and processor-held copies were located.

2. The controller assessed whether Article 17 applied and recorded any applicable exception under Article 17(3).

3. The controller selected a sanitisation method appropriate to the storage medium, the sensitivity of the data, the threat model, and the intended disposal or reuse of the equipment.

4. The deletion operation produced an authenticated record identifying the relevant asset, the deletion scope, the method used, the time of execution, the responsible process or operator, and the result reported by the storage system.

5. The controller performed verification proportionate to the risk. Depending on the environment, this may include device-level logs, cryptographic-key destruction records, confirmation from processors, controlled recovery attempts, independent sampling, or a certificate of physical destruction.

6. The controller documented limitations and exceptions, including failed commands, interrupted operations, inaccessible storage areas, cloud replicas, offline backups, retention periods, and data that could not yet be deleted because of a legal obligation.

2.5.4 Technical Challenges to the Right to Be Forgotten

The tension between the right to be forgotten and technical system design is a recurring theme in the literature. Visconti (2024) examines the Right to Be Forgotten (RtBF) through a cryptographic lens, identifying challenges such as the avalanche effect, where removing a single piece of personal data creates inconsistencies in correlated data stored on a digital platform [**Visconti2024zeroknowledge**]. Traditional cryptographic tools are insufficient to enforce GDPR compliance, and the gap between the language of the regulation and the technical specifications required by developers is wide enough to introduce significant uncertainty. Visconti shows that zero-knowledge (ZK) proofs can bridge this gap by allowing digital platforms to prove compliance with deletion requests without revealing the underlying personal data, and proposes an assessment scheme applicable to use cases including machine unlearning and redactable blockchains [**Visconti2024zeroknowledge**].

At the standards level, Aronsson and Lund (2023) review NIST SP 800-88, IEEE 2883-2022, and NATO AEDP-3, all of which structure deletion methods by security level. These standards categorize sanitization into Clear and Purge levels, where Purge renders data infeasible to recover through block erase or cryptographic erase, and Destruct covers physical destruction [**aronsson2025secure**]. Overwriting methods such as scrubbing are not regulation-approved, which limits their practical applicability despite promising performance and security characteristics.

2.5.5 Digital Evidence and Cross-Border Procedure

The handling of digital evidence across borders intersects directly with secure deletion obligations. Casino et al. (2022), in their systematization of digital forensics research, identify that one of the most reported challenges across the field is the lack of jurisdictional and legal requirements for different investigation scenarios, particularly regarding ethics and data management of confidential and personal data [**Casino2022Research**]. Distributed systems such as cloud and blockchain mean that software and data may reside in different countries, requiring specific cross-border collaborations. A proper understanding between all actors

involved, including stakeholders, law enforcement agencies, and court members, is mandatory to ensure successful prosecution, with the chain of custody remaining a persistent challenge [**Casino2022Research**].

Ontanu (2023) examines the normative framework for electronic evidence in EU civil procedure, finding that while regulations such as the GDPR, eIDAS, and the revised Taking of Evidence Regulation address some aspects, there is little guidance on metadata, technical standards, or requirements for electronic evidence [**Ontanu2023**]. The evaluation of electronic evidence is left to domestic legislation under principles of procedural autonomy, leaving significant uncertainty about how deletion obligations apply to evidentiary materials across member states. Allah Rakha (2024) addresses jurisdiction and governance in the global digital economy, arguing that ambiguous jurisdiction enables large platforms to circumvent laws while prescriptive control risks stifling innovation, and calls for cooperative solutions aligned to collective rights and welfare [**AllahRakha2024**].

2.5.6 Comparative Regulatory Landscape

Birrell et al. (2024) provide the broadest cross-jurisdictional analysis, examining 25 Internet privacy and data protection regulations from around the world and developing a taxonomy of rights and obligations. Their work systematizes 270 technical research papers, finding that the overwhelming majority study either the GDPR or the CCPA, laws protecting approximately 6.3% of the world's population, while other aspects such as the right to non-discrimination remain largely unexplored [**Birrell2024SoK**]. They identify significant technical challenges to implementing the right to delete, including the difficulty of deleting all replicas of data across distributed data centers, backups, and offline copies, with some systems taking months to complete deletion. The right to erasure under GDPR is the most widely known of the self-management rights, yet users are often not aware of or do not use deletion controls due to difficulties locating them or encountering dark patterns [**Birrell2024SoK**].

Beyond the GDPR, the California Consumer Privacy Act (CCPA) similarly confers a right to deletion, and sectoral laws such as Sarbanes-Oxley mandate retention periods after which secure deletion becomes necessary to limit liability [**AlgorithmicDestruction**]. The technical literature contextualizes these frameworks directly: the Holepunch system explicitly frames secure deletion via cryptographic erasure as a promising mechanism for enforcing GDPR-style regulations, provided that challenges involving key management, performance, and crash consistency can be solved [**holepunch**].

Bridging Technical and Legal Perspectives

The literature reveals both alignment and tension between the technical capabilities of secure deletion and the legal requirements imposed by data protection regulations.

On the alignment side, cryptographic erasure offers a technically mature approach that maps well to regulatory requirements. The ability to render data irrecoverable through key deletion, independent of the underlying storage medium, provides a mechanism that can satisfy right-to-erasure obligations across diverse technical environments [**reardon2013sok**]. The key management architectures developed in systems such as Eraser, DNEFS, and Holepunch demonstrate that file-granularity deletion is achievable with acceptable performance overhead [**holepunch**].

On the tension side, regulatory mandates require timely and demonstrable deletion, but technical constraints often prevent this. Large cloud providers describe deletion as an iterative process that can take up to 180 days due to data replication and backup cycles [**Birrell2024SoK**], far exceeding the timelines implied by Article 17 of the GDPR. The six GDPR anti-patterns identified by Shastri et al. (2019) in cloud-scale systems — storing data without clear deletion timelines, reusing data indiscriminately, and risk-agnostic data processing, among others — suggest that achieving regulatory compliance requires systemic architectural changes rather than point solutions [**Shastri2019gdpr**].

The gap between legal language and technical specification remains a central challenge. Visconti (2024) demonstrates that the gap between the language of the GDPR and the technical specifications required by developers is wide enough to introduce significant uncertainty, and proposes zero-knowledge proofs as a bridge [**Visconti2024zeroknowledge**]. Cross-border procedure adds further complexity, as Casino et al. (2022) note that distributed systems mean software and data may reside in different countries, requiring specific cross-border collaborations that existing deletion frameworks do not adequately address [**Casino2022Research**]. Standardized evaluation frameworks such as NIST SP 800-88 Rev. 1 and IEEE 2883-2022 provide operational definitions for sanitization levels but remain limited by their focus on traditional storage media [**aronsson2025secure**], leaving emerging contexts such as containers, cloud storage, and AI systems without clear compliance guidance.

Chapter 3

Experimental Framework and Methodology

This chapter presents the experimental framework developed to evaluate secure deletion techniques in modern storage environments. The framework is designed to assess not only whether a deletion technique prevents data recovery, but also how quickly it operates, how consistently it behaves across storage devices, and what limitations affect verification from the host system.

Secure deletion is difficult to evaluate because the deletion request issued by the operating system does not necessarily correspond to the physical removal of data from the storage medium. In NAND flash storage, the Flash Translation Layer (FTL) maps logical block addresses to physical pages and performs operations such as wear leveling, garbage collection, and out-of-place updates. These processes may create or preserve physical copies of data that are no longer visible through the file system. Consequently, a file may appear to have been deleted at the file-system level while remnants remain recoverable at lower storage layers.

Previous experimental studies have used different devices, workloads, deletion procedures, recovery methods, and performance metrics. This variation makes it difficult to compare findings across studies. The literature therefore identifies the absence of a standardized and reproducible evaluation framework as a major methodological gap. The framework proposed in this chapter addresses this gap by defining common experimental conditions, measurable evaluation criteria, repeatable testing procedures, and a host-side verification methodology.

The study focuses on outcomes that can be observed from the host perspective. It does not assume that a command completion message from the operating system or storage controller proves physical destruction. Instead, secure deletion is evaluated using observable indicators such as post-deletion data recovery, residual data persistence, deletion latency, performance variability, and differences between

storage devices.

3.1 Research Objectives

The aim of this thesis is to develop an experimental framework to evaluate the effectiveness, performance, and practical limitations of secure deletion techniques in modern storage environments. Although many secure deletion mechanisms have been proposed in the literature, their behavior in real-world systems is difficult to verify because of the abstraction layers introduced by file systems, flash translation layers (FTLs), and storage controllers. This work therefore focuses on outcomes that are observable and measurable from the host perspective.

The objectives of this study are:

1. To evaluate the effectiveness of different secure deletion techniques in preventing post-deletion data recovery.
2. To investigate how long deleted data persists after TRIM and sanitization operations, and to assess the practical implications of data remanence.
3. To compare the latency characteristics and variability of secure deletion mechanisms, including TRIM, block erase, and cryptographic erase operations.
4. To examine how filesystem and storage-layer behavior, such as journaling and FTL remapping, influences the effectiveness of secure deletion.
5. To analyse differences in secure deletion behavior across consumer and enterprise storage devices.
6. To identify the practical limitations of host-side verification and establish a methodology for evaluating secure deletion techniques under realistic operating conditions.

3.1.1 Research Questions

To achieve these objectives, the following research questions are investigated:

RQ1: To what extent do different secure deletion techniques prevent the recovery of previously stored data from a host-side forensic perspective?

RQ2: How does the recoverability of deleted data change over time following TRIM and sanitization operations?

RQ3: What are the latency characteristics and performance variability of TRIM operations across different storage devices and filesystem utilization levels?

RQ4: How do filesystem mechanisms and flash translation layer behavior affect the effectiveness of secure deletion techniques?

RQ5: How does secure deletion behavior vary across different SSD implementations and vendors?

RQ6: What are the practical limitations of verifying secure deletion from the host side, and how can these limitations be incorporated into an experimental evaluation framework?

3.2 Experimental Environment

Host System

The experiments were conducted using an Acer Nitro AN515-58 laptop operating as the host system. The system ran Ubuntu 26.04 LTS with a Linux 7.0.0-28-generic kernel on an x86-64 architecture. The host system firmware version was recorded as V2.21. The use of a consistent host operating system and kernel was intended to reduce variation in file-system behaviour, device-driver operation, command execution, and communication with the storage devices. The host system provided the operating environment through which the deletion commands, device-inspection utilities, system-management commands, and storage-analysis procedures were executed. The experiments were performed using administrative privileges where required because device-level inspection and sanitization commands cannot normally be executed by an unprivileged user.

Test Drives

Two 128 GB solid-state storage devices were used to conduct the experiments. The devices were chosen to represent different storage interfaces and controller communication methods. One device used the NVMe interface, while the second device was a 2.5-inch SATA solid-state drive. The first device was a Phison E4c PCLe NVMe. It used NVMe version 1.4, had a reported capacity of approximately 128 GB, and used a formatted logical block size of 512 bytes. Its recorded firmware version was HPE4BP01. The device reported support for Data Set Management commands and Security Format operations. This device will be referred to as the **NVMe SSD** in the remainder of this thesis. The NVMe SSD was connected to the host system using a JMicron JMS583 USB 3.0 enclosure.

The second device was identified as an EDILOCA ES106 128GB solid-state drive. It used a Serial ATA interface and was reported as compatible with SATA standards up to SATA Revision 3.0. The device had a capacity of approximately 128 GB, a 2.5-inch form factor, and a firmware revision of HC2705H3. Its recorded capabilities included the ATA SANITIZE feature set, the BLOCK ERASE EXT command, Data Set Management TRIM support, and enhanced security erase support. The two devices were used to examine whether differences in storage

interface, device implementation, firmware, and supported sanitization commands affected deletion latency and post-deletion data recoverability. The devices were not treated as interchangeable. Results were recorded separately for each model because storage behaviour varied between controller implementations, firmware versions, capacities, and device interfaces. This device will be referred to as the **SATA SSD** in the remainder of this thesis and it was connected to the host system using a USB 3.0 to SATA adapter.

The connection path between the host system and each SSD formed an additional component of the experimental environment. Since the NVMe SSD was not compatible with the host system's M.2 slot and the native SATA port was already occupied, neither storage device was connected directly to the native storage interface of the host. The NVMe SSD was connected through a JMicron JMS583 USB 3.0 enclosure, while the SATA SSD was connected through a USB 3.0 to SATA adapter. This arrangement introduces a possible translation layer between the operating system and the SSD controller. Depending on the implementation of the enclosure or adapter, a host-level discard request may be translated into an equivalent command, partially forwarded, delayed, modified, or not forwarded to the SSD. In the case of USB-attached storage, the host may communicate with the bridge using a command such as SCSI UNMAP, while the bridge is responsible for translating that request into the command format understood by the underlying SATA or NVMe device.

The influence of this connection path became evident during the preparation of the NVMe experiment. Ubuntu did not initially recognise discard support through the USB enclosure. The enclosure required an explicit configuration that enabled the relevant unmap behaviour. After the configuration was applied, the discard capability was verified using the reported discard-granularity and discard-maximum values. The verification confirmed that the host system exposed a discard capability through the connection path. However, this verification did not independently demonstrate that every discard request was received and processed by the SSD controller, nor did it confirm that the controller physically erased the corresponding NAND pages. The reported capability should therefore be interpreted as evidence of host-visible command support rather than proof of end-to-end TRIM or UNMAP execution. This issue is particularly relevant when comparing the NVMe and SATA results. Differences in recovery may have resulted from the SSD controller and firmware, but they may also have been influenced by the USB enclosure, the USB-to-SATA adapter, command translation, driver behaviour, or the manner in which each bridge handled discard requests. The connection path was therefore retained as a methodological limitation and treated as a potential source of variability.

3.3 Experiments Conducted

3.3.1 Experiment 1: TRIM and Filesystem-Level Data Recoverability

This experiment was conducted to investigate the effect of the TRIM operation on the recoverability of deleted data at the filesystem level. The objective was to determine whether information associated with a deleted file remained accessible through filesystem metadata, inode structures, the ext4 journal, or the previously allocated data blocks after TRIM. The experiment was performed on an SSD formatted with the ext4 filesystem using a block size of 4096 bytes. A test file was created and its filesystem metadata and block allocation information were recorded before deletion. The relevant inode and journal structures were subsequently examined to establish the state of the filesystem before and after deletion and TRIM.

A 100 MB test file containing random, non-compressible data was created on an ext4 filesystem using a 4096-byte block size. The inode number, file size, block allocation, filesystem journal state, and superblock information were recorded before deletion. The test file was then deleted using the rm command, followed by synchronisation and an explicit fstrim operation. The following tools were used for analysis:

- ext4magic: A recovery tool capable of extracting deleted files from ext3/ext4 filesystems using journal data
- debugfs: A filesystem debugger for examining ext4 internal structures
- dumpe2fs: A tool for displaying filesystem superblock and group descriptor information
- dd with xxd: For direct block-level inspection of allocated data blocks

The investigation consisted of four complementary stages. First, the inode was examined using debugfs. The analysis recorded whether the inode remained allocated, whether the block pointers were still present, whether the size and block count had changed, and whether the inode remained associated with a pathname. Second, the ext4 journal was examined using dumpe2fs and debugfs. The purpose was to determine whether the deletion operation had been recorded in the journal and whether the journal contained file data or only metadata updates. Third, recovery was attempted using ext4magic. The tool was directed to the relevant inode and, in a secondary attempt, to the journal start block. This allowed the experiment to test whether the journal contained sufficient information to reconstruct the deleted file. Fourth, the previously allocated data blocks were

read directly using dd and examined with hexadecimal output. Individual journal blocks were also extracted so that the inode state recorded in the journal could be compared with the current filesystem state. The purpose of using several methods was to avoid relying on one indicator. A filesystem-level recovery failure alone might reflect a limitation of the recovery tool. Similarly, an empty inode alone would demonstrate logical deallocation but would not establish the state of the underlying NAND cells. Combining metadata analysis, journal inspection, recovery attempts, and direct block inspection produced a more complete host-side assessment.

3.3.2 Experiment 2: Data Recovery After TRIM

This experiment investigated the effect of TRIM configuration on the host-side recoverability of deleted data from the tested NVMe and SATA SSDs. The experiment compared the number of dataset-specific signatures recovered under TRIM-enabled and TRIM-disabled conditions using a forensic recovery procedure. The experiment was conducted on the Ubuntu host machine using and tests were performed on the NVMe and SATA SSDs. Before each experimental run, the beginning of the storage device was overwritten with zeros to remove existing partition and filesystem structures and the device was prepared with a new GPT partition table and ext4 filesystem.

Discard behaviour was controlled using the filesystem mount options `discard` and `nodiscard`. The TRIM-enabled condition used the discard mount option, while the TRIM-disabled condition used nodiscard. In the TRIM-enabled condition, an explicit fstrim operation was also performed after deletion.

The experiment was repeated five times for each storage configuration. The repeated runs were intended to assess whether the results were reproducible or whether the storage devices produced variable outcomes under nominally identical conditions. Each run used the same dataset allocation, deletion procedure, recovery method, and signature-counting procedure.

The experiment therefore treated the following as independent or controlled factors:

- SSD type, consisting of NVMe and SATA devices.
- TRIM configuration, consisting of the nominal TRIM-enabled and TRIM-disabled conditions.
- Dataset status, consisting of kept, deleted, and post-deletion write datasets.
- Experimental run, consisting of five repeated runs for each storage configuration.
- Connection path, consisting of the USB-NVMe enclosure and the USB-SATA adapter.

The connection path was included as an important methodological variable because neither SSD was connected directly through a native PCIe or SATA interface. The NVMe device was accessed through a JMicron JMS583 USB enclosure, while the SATA device was accessed through a USB-to-SATA adapter. The host-level confirmation did not independently prove that every discard request was correctly forwarded to the SSD controller. It confirmed only that the operating system exposed a discard-capable path. The effective TRIM condition should therefore be understood as the behaviour of the complete host, driver, USB transport, bridge, and SSD configuration.

Dataset Generation and Allocation

To enable precise measurement of data recovery, a custom data generation script was developed. The script created unique, searchable signatures within each dataset, allowing recovery success to be quantified by counting the number of signatures found after deletion. Each dataset used a dataset-specific signature identifier, allowing recovered content to be attributed to its original dataset and enabling recovery to be quantified consistently across experimental conditions.Each dataset contained repeated-signature text files, multiple small text files, binary files containing embedded signatures, and checksum information. The mixture was intended to test recovery of both text-based evidence and binary content.

The experiment included three categories of datasets:

- Kept datasets served as positive controls and were not deleted.
- Deleted datasets formed the main test group.
- Post-deletion write datasets were created after the deletion of the test group to evaluate whether subsequent logical writes removed the signatures associated with the earlier datasets.

The dataset allocation included three kept datasets, two deleted datasets, and three post-deletion write datasets for each TRIM condition. This arrangement allowed the recovery procedure to be checked using positive controls while also testing both deletion and later write activity.

Table 3.1: Dataset Allocation for the TRIM Recovery Experiment

Dataset	TRIM	Status	Purpose
DS1	ON	Kept	Positive control
DS2	ON	Deleted	Test group
DS3	ON	Kept	Positive control
DS4	ON	Deleted	Test group
DS5	ON	Kept	Positive control
DS6–8	ON	Post Deletion Write	Test effect of later writes
DS9	OFF	Kept	Positive control
DS10	OFF	Deleted	Test group
DS11	OFF	Kept	Positive control
DS12	OFF	Deleted	Test group
DS13	OFF	Kept	Positive control
DS14–16	OFF	Post Deletion Write	Test effect of later writes

This design ensured that for each SSD type and TRIM status, there were 3 kept datasets (positive controls) to confirm recovery tools worked, 2 deleted datasets (test groups) to measure recovery after deletion and 3 post-deletion write datasets to test whether new data destroyed old data

For each run, the following procedure was applied:

1. The SSD was prepared and formatted with an ext4 filesystem.
2. The relevant nominal TRIM condition was configured.
3. Datasets 1 to 5 were created, and datasets 2 and 4 were deleted.
4. The nominal TRIM-enabled condition received an explicit fstrim operation.
5. Datasets 6 to 8 were created after the deletion operation.
6. Datasets 9 to 13 were created under the nominal TRIM-disabled condition.
7. Datasets 10 and 12 were deleted, and no explicit fstrim operation was issued in that condition.
8. Datasets 14 to 16 were created after deletion.
9. The device was scanned using the selected recovery tools.
10. Dataset-specific signature occurrences were counted and recorded for each dataset.

Recovery Procedure

PhotoRec was used to attempt recovery of deleted datasets. PhotoRec is a widely used open-source data recovery tool that can recover lost files from various storage devices. The recovery process involved scanning the entire SSD for recoverable files and searching for the unique signatures embedded in the datasets. PhotoRec performes full file carving based on file headers and footers and is able to recover complete files regardless of the filesystem. It is interactive and requires user input to specify the source device, file types to recover, and destination for recovered files. PhotoRec was configured to scan the whole partition rather than only unallocated space. Scanning the whole partition is necessary when the filesystem may be corrupted or when deleted data may reside in areas not marked as unallocated . This approach maximises the chance of recovering residual data from previously allocated blocks. And Recovered files were written to a separate storage device within the host system's internal storage to prevent overwriting the target SSD

Since PhotoRec cannot recover original filenames or filesystem metadata , identification of recovered content relied entirely on the embedded signatures. After PhotoRec completed the recovery process, the recovered files were analyzed to determine the presence of the unique signatures from the deleted datasets. The number of successfully recovered signatures was recorded for each dataset, allowing for a quantitative assessment of data recoverability under different TRIM conditions.

Recovery was quantified using the number of dataset-specific signature occurrences identified in the recovered output. For each dataset, the recovery rate was calculated as:

$$\text{Recovery Rate (\%)} = \frac{\text{Recovered Signatures}}{\text{Expected Signatures}} \times 100$$

A recovery rate of 100% indicates that all expected signature occurrences were identified in the recovery output, whereas a recovery rate of 0% indicates that no matching signatures were identified. The metric represents host-side recovery by the specified forensic procedure and should not be interpreted as a direct measurement of the physical state of NAND flash. Recovery rates were calculated as the mean number of recovered signatures across the five runs, with standard deviation reported to quantify variability.

The same experimental procedure, dataset-generation script, TRIM configuration, deletion procedure, and recovery procedure were applied to both storage devices. Results were recorded separately for each device to account for differences in device implementation and controller behaviour.

3.3.3 Experiment 3: Testing Recoverability at different fill levels and times intervals

Experiment 2 was repeated with the addition of a time delay between deletion and recovery to investigate how recoverability changed over time. The experiment was conducted at different fill levels of the SSD to assess whether the amount of free space affected the persistence of deleted data. The experiment also recorded the execution time of the fstrim operation in order to characterize the host-visible latency associated with issuing discard requests to the storage device. An automated script was used to automate the process of wiping the drive,creating datasets, deleting them, waiting for a specified time interval, and then attempting recovery.

The experiment was performed using a controlled test environment in which the storage device was formatted with the ext4 filesystem. Before each experimental sequence, the device was unmounted and the beginning of the device was overwritten with zeros before a new GPT partition table and ext4 filesystem were created. The experimental filesystem was subsequently mounted at the designated test mount point. The experimental procedure therefore began from a freshly initialized filesystem state, although the initial 100 MiB overwrite did not constitute a complete physical wipe of the entire SSD.

Prior to the recovery experiment, the filesystem was filled to a specified target utilisation level. The amount of filler data was calculated from the filesystem capacity and the selected fill percentage, after which random data was written in chunks of up to 100 MB until the target utilisation level was reached. The resulting filesystem utilisation was recorded before the experimental dataset was created. A dataset of approximately 2.5 GB was then generated for each experimental iteration. Each dataset consisted of multiple file types, including text files, a large ZIP archive, numerous small text files, a JPEG image, and a PDF document. Dataset-specific identifiers were embedded within the generated files to enable the experimental data to be distinguished from unrelated recovered content. The dataset also included a checksum file generated from the experimental files. This provided a reference for the original dataset contents and supported subsequent verification of recovered data.

For each iteration, a unique dataset directory was created to ensure that the experimental data could be distinguished across iterations. The dataset was first created and its total size and file count were recorded. The complete dataset was subsequently deleted using rm -rf, followed by sync and a short five-second delay. After deletion, the filesystem was subjected to an explicit fstrim operation. Before measuring the operation, filesystem caches were flushed and the Linux page cache was cleared. The fstrim command was executed using /usr/bin/time -v, allowing the elapsed, user and system execution times to be recorded. The amount of storage reported as trimmed by the command was also recorded. Following

the TRIM operation, forensic recovery was attempted after a predefined waiting period. The experiment used different recovery delays to examine whether the passage of time following TRIM affected the amount of data observable through the recovery procedure. The configured waiting periods were 0, 5, 15 and 30 minutes. At each recovery point, the foremost forensic recovery tool was executed against the test partition using all supported file types. The number of recovered files was then recorded in a CSV results file together with the iteration number, waiting period, filesystem fill level, measured TRIM execution times and the amount of data reported as trimmed.

The experiment recorded the following variables for each iteration:

1. **Filesystem fill level**: representing the proportion of the filesystem occupied before the experimental dataset was created.
2. **Recovery delay**: representing the time between completion of TRIM and the beginning of the forensic recovery attempt.
3. **TRIM execution time**: including real, user and system time reported by /usr/bin/time.
4. **Amount of storage reported as trimmed**: by fstrim.
5. **Number of recovered files**: as reported by the recovery output.

The primary relationship investigated was therefore between the recovery delay following TRIM and the amount of data recovered by the forensic tool. The execution time represents the host-visible duration of the discard operation and does not provide direct evidence of when the SSD controller performs internal garbage collection, block erasure or other flash-management operations. Consequently, the latency measurement is interpreted as host-observable TRIM latency, rather than as a direct measurement of physical data-erasure latency. For each recovery delay, the recovered output was examined and the number of recovered files was recorded. The results were then compared across the different waiting periods.

The experiment was repeated under Trim off conditions to provide a baseline for comparison. In the TRIM-disabled condition, the same dataset creation, deletion, and recovery procedures were applied, but no fstrim operation was performed. The filesystem was filled to the same target utilisation levels, and the same recovery delays were applied. The recovered output was analyzed in the same manner, and the number of recovered files was recorded for each recovery delay. This allowed a direct comparison of the recoverability of deleted data under TRIM-enabled and TRIM-disabled conditions, as well as an assessment of how recovery changed over time following deletion. Five seperate runs were performed for each storage device, fill level, and TRIM condition. The repeated runs were intended to assess whether

the results were reproducible or whether the storage devices produced variable outcomes under nominally identical conditions. Each run used the same dataset allocation, deletion procedure, and recovery method.

3.3.4 Experiment 4: Cryptographic Erase Attempt

Following the TRIM and recovery experiments, an attempt was made to test drive-level sanitization using the ATA command set. The objective was to evaluate whether cryptographic erase and block erase sanitization commands could be executed through the tested connection path, and to compare their effectiveness against TRIM-based deletion.

Two command sets were attempted:

1. **ATA Security Erase** (`hdparm -security-erase`), which invokes the legacy ATA Security feature set.
2. **ATA SANITIZE** (`hdparm -sanitize-crypto-scramble` and `hdparm -sanitize-block-erase`), which invokes the newer `SANITIZE` feature set introduced in ATA ACS-4.

Before each attempt, the drive's identify data was examined using `hdparm -I` to confirm support for the relevant feature sets. The SATA SSD reported support for both the `Security Mode` feature set and the `SANITIZE` feature set, including `BLOCK_ERASE_EXT`.

The commands were issued with the `-yes-i-know-what-i-am-doing` flag, which is required by `hdparm` to acknowledge the destructive nature of the operation. Status queries were attempted using `hdparm -sanitize-status` to determine whether any sanitize operation had been accepted by the drive.

Chapter 4

Experimental Results and Analysis

4.1 Results Analysis

This chapter presents the results obtained from the three experiments. The findings are reported separately for filesystem-level inspection, TRIM-enabled and TRIM-disabled recovery conditions, storage-device type, filesystem utilisation, recovery delay, and TRIM execution time. The results are interpreted using the distinction between logical deletion, host-side recoverability, and physical sanitization. A file may disappear from the filesystem while data remains in storage areas that cannot be observed through ordinary host-level access. Similarly, an absence of recovered signatures indicates that the selected recovery procedure did not identify the data, but it does not independently prove that all physical copies have been erased.

4.1.1 Experiment 1: TRIM and Filesystem-Level Data Recoverability

Inode State After Deletion and TRIM

The inode examination showed that the deleted file had been logically removed from the ext4 filesystem. The inode reported a size of zero and a block count of zero. Its link count was also zero, indicating that no directory entry referenced the inode. The extent tree was empty, showing that no usable block pointers remained in the inode. The debugfs stat command returned the following inode information after deletion:

```
Inode: 6291458   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 2405750640    Version: 0x00000000:0000005c
```

```
User:     0   Group:     0   Project:     0   Size: 0
File ACL: 0
Links: 0   Blockcount: 0
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6a86c9f3:ad975988 -- Thu Aug 20 11:33:39 2026
 atime: 0x6a86c7cc:115e960c -- Thu Aug 20 11:24:28 2026
 mtime: 0x6a86c9f3:ad975988 -- Thu Aug 20 11:33:39 2026
 crtime: 0x6a86c7cc:115e960c -- Thu Aug 20 11:24:28 2026
 dtime: 0x6a86c9f3:(ad975988) -- Thu Aug 20 11:33:39 2026
Size of extra inode fields: 32
Inode checksum: 0xbe4252c1
EXTENTS:
```

Several observations from the inode state are significant:

- **Size and Blockcount:** Both were zero, indicating that the file content had been logically unlinked from the inode. The Size: 0 and Blockcount: 0 fields confirm that the inode no longer references any data blocks.
- **Links:** The Links: 0 value indicates that no directory entries reference this inode, confirming it was unlinked.
- **Deletion Timestamp:** The dtime field was set to the deletion time (Thu Aug 20 11:33:39 2026), confirming that the inode had been deallocated.
- **EXTENTS:** The extent tree was empty, meaning no block pointers were retained in the inode.
- **Inode Type and Flags:** The inode remained marked as type regular with flags 0x80000, but the loss of block pointers made data recovery through filesystem metadata impossible.

The deletion timestamp was recorded in the inode. This timestamp provides forensic evidence that the inode had been deallocated, but it does not provide a mechanism for recovering the file content. The inode remained identifiable as a regular file, but its filesystem references to data blocks had been removed. These observations demonstrate successful logical deletion at the filesystem level. They do not, by themselves, establish whether copies of the file remained in physical NAND pages, remapped blocks, over-provisioned regions, or controller-managed areas.

Journal State Comparison

The journal state before and after deletion showed an important distinction. The journal checksum changed after deletion while the journal sequence number remained unchanged. This indicates that deletion-related metadata had been written to the journal. The journal therefore preserved evidence of filesystem activity associated with deletion.

Before deletion:

```
Journal sequence: 0x0000003d
Journal checksum: 0x0cd525e3
```

After deletion:

```
Journal sequence: 0x0000003d
Journal checksum: 0x92f1d2b1
```

ext4magic Recovery Attempt

The ext4magic recovery produced the following output:

```
Filesystem in use: /dev/sda1

Dump internal Inode 6291458
Status : Inode is Unallocated

Inode: 6291458   Type: regular    Mode:  0644   Flags: 0x80000
...
ext4magic : EXIT_SUCCESS
```

The tool reported that the inode was unallocated (Status : Inode is Unallocated) and could not recover the file content. Even when the journal start block was specified, ext4magic was unable to reconstruct the file. The output "Dump Filesystemblock 620 Status : Block is Allocated" indicated that the journal block itself was present but contained no recoverable file data.

Pathname Resolution

The ncheck command returned no pathname for the inode:

```
debugfs: ncheck 6291458
Inode Pathname
debugfs: q
```

This confirmed that no directory entry pointed to the inode, consistent with the file having been unlinked.

Inode Map Location

The imap command identified the inode's location in the filesystem:

```
Inode 6291458 is part of block group 768
    located at block 25165856, offset 0x0100
```

This information confirmed that the inode was physically present but its contents had been cleared.

Journal Log Analysis

The journal logdump revealed the transactions that had been written during the deletion process:

```
Journal starts at block 620, transaction 61
Found expected sequence 61, type 1 (descriptor block) at block 620
Found expected sequence 61, type 2 (commit block) at block 626
Found expected sequence 62, type 1 (descriptor block) at block 627
Found expected sequence 62, type 2 (commit block) at block 633
Found expected sequence 63, type 1 (descriptor block) at block 634
Found expected sequence 63, type 2 (commit block) at block 637
Found expected sequence 64, type 5 (revoke table) at block 638
Found expected sequence 64, type 1 (descriptor block) at block 639
Found expected sequence 64, type 2 (commit block) at block 652
No magic number at block 653: end of journal.
```

The journal contained four transactions (61–64) related to the deletion. Notably, transaction 64 included a revoke table (type 5), which is used to indicate that certain blocks should no longer be considered valid in the journal. This revocation mechanism is relevant to filesystem-level recoverability after deletion.

The detailed descriptor block dump showed which filesystem blocks were logged:

```
Dumping descriptor block, sequence 64, at block 639:
  FS block 25174048 logged at journal block 640 (flags 0x0)
  FS block 25165856 logged at journal block 641 (flags 0x2)
  FS block 9504 logged at journal block 642 (flags 0x2)
  FS block 25165825 logged at journal block 643 (flags 0x2)
  FS block 13 logged at journal block 644 (flags 0x2)
  FS block 2097166 logged at journal block 645 (flags 0x2)
  FS block 2 logged at journal block 646 (flags 0x2)
  FS block 25165840 logged at journal block 647 (flags 0x2)
  FS block 26214432 logged at journal block 648 (flags 0x2)
```

```
FS block 30933001 logged at journal block 649 (flags 0x2)
FS block 15 logged at journal block 650 (flags 0x2)
FS block 26214416 logged at journal block 651 (flags 0xa)
```

The journal blocks included inodes and metadata blocks but did not contain the actual file data blocks, which explains why ext4magic could not recover the file content.

Direct Block Inspection

The direct inspection of previously allocated data blocks showed that they contained zero bytes:

```
sudo dd if=/dev/sda1 bs=4096 skip=33388 count=1 2>/dev/null| xxd| head -20
00000000: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
```

Both previously allocated blocks (33388 and 33389) contained only zeros. This indicates that the data blocks had been sanitized—either through the TRIM operation itself, through subsequent garbage collection, or through the filesystem's block deallocation process that returned them to the free list.

Journal Inode Extraction

The extracted inode portions from the journal blocks confirmed that no recoverable inode data remained:

```
sudo dd if=~/inode_analysis/journal_tx61.bin
of=~/inode_analysis/inode_tx61.bin bs=1 skip=256 count=256
```

The extracted journal inode showed that the journal recorded the inode in its deallocated state, with zero size and block count, consistent with the debugfs stat output.

The experimental results demonstrate several important characteristics of filesystem-level recoverability after deletion and TRIM. The inode state after deletion shows that ext4 fully deallocates the inode when a file is removed. The inode's block pointers are cleared (Blockcount: 0, empty EXTENTS), the size is set to zero (Size: 0), and the link count is set to zero (Links: 0). The dtime field records the deletion time, which is a forensic indicator but does not assist in data recovery. This deallocation pattern is consistent with the standard ext4 deletion behavior described in the filesystem documentation: when a file is unlinked, the inode is marked as free and its block pointers are cleared to prevent the filesystem

from treating stale block references as valid. The journal analysis reveals that the deletion was recorded in the journal, but the journal contained only metadata updates, not the actual file data. This is expected behavior for the ext4 filesystem in its default data-ordered mode: metadata is journaled before it is written to the filesystem, but data blocks are not necessarily journaled. Consequently, while the journal can be used to recover filesystem metadata after a crash, it does not provide a reliable mechanism for recovering deleted file data. The revoke table in transaction 64 is particularly significant: it indicates that the journal explicitly invalidated certain blocks, further reducing the likelihood of recovery through journal replay. The direct block inspection showed that the previously allocated data blocks contained zero bytes. This is consistent with the TRIM operation having informed the SSD that the logical blocks were no longer in use, allowing the controller to discard the physical data. The zeros observed in the block inspection could reflect either physical erasure of the NAND cells or the controller returning zeroed data at the logical block level. Either interpretation is consistent with the conclusion that the data was not recoverable through host-side block access.

4.1.2 Experiment 2: Data Recovery After TRIM

Experiment 2 evaluated whether deleted data could be recovered from the NVMe and SATA SSDs under nominal TRIM-enabled and TRIM-disabled conditions. Recovery was assessed by counting dataset-specific signatures identified in the PhotoRec output. Each dataset contained 5,052 expected signatures, and five repeated runs were conducted for each storage configuration. The results are presented separately for the NVMe and SATA SSDs because the two devices produced different recovery patterns

NVMe SSD Results

Table 4.1: Data Recovery Results After TRIM on NVMe SSD

Dataset	Signatures Recovered				
	Run 1	**Run 2**	**Run 3**	**Run 4**	**Run 5**
DS1	5,052	5,052	5,052	5,052	5,052
DS2	0	0	0	0	0
DS3	5,052	5,052	5,052	5,052	5,052
DS4	0	0	0	0	0
DS5	5,052	5,052	5,052	5,052	5,052
DS6	5,052	5,052	5,052	5,052	5,052
DS7	5,052	5,052	5,052	5,052	5,052
DS8	5,052	5,052	5,052	5,052	5,052
DS9	5,052	5,052	5,052	5,052	5,052
DS10	5,052	5,052	5,001	5,001	0
DS11	5,052	5,052	5,052	5,052	5,052
DS12	5,052	5,052	5,052	0	0
DS13	5,052	5,052	5,052	5,052	5,052
DS14	5,052	5,052	5,052	5,052	5,052
DS15	5,052	5,052	5,052	5,052	5,052
DS16	5,052	5,052	5,052	5,052	5,052

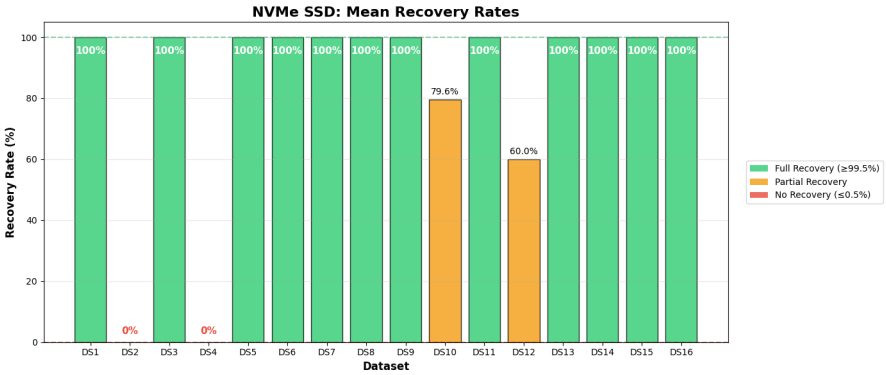


Figure 4.1: Mean Recovery on NVMe SSD

The NVMe results showed that the recovery procedure was effective for datasets that remained available on the device. The kept datasets, DS1, DS3, DS5, DS9, DS11, and DS13, each produced 5,052 recovered signatures in all five runs. This

represents a recovery rate of 100% for every kept dataset. These results provide a positive control because they demonstrate that the recovery tool was able to scan the device and identify the embedded dataset signatures when the data had not been removed.

The post-deletion write datasets, DS6 to DS8 and DS14 to DS16, also produced 5,052 recovered signatures in every run. Their recovery rates were therefore 100%. The recovery of these datasets confirms that the later write stage produced data that could be identified by the recovery procedure. However, their recovery should not be interpreted as recovery of the earlier deleted datasets. Instead, these datasets functioned as a separate test of whether newly written data remained observable after the deletion stage.

The results for the deleted datasets under the nominal TRIM-enabled condition were considerably different. DS2 and DS4 produced zero recovered signatures in every run. Consequently, both datasets recorded a mean recovery rate of 0%. The consistency of this result across all five repetitions indicates that no dataset-specific signatures from DS2 or DS4 were identified after deletion and the subsequent TRIM operation.

The two deleted datasets under the nominal TRIM-disabled condition produced substantially higher recovery rates. DS10 produced 5,052 signatures in the first two runs, 5,001 signatures in the third and fourth runs, and zero signatures in the fifth run. The mean number of recovered signatures was 4,021.2, corresponding to a mean recovery rate of 79.60%. DS12 produced 5,052 signatures in each of the first three runs and zero signatures in the final two runs. Its mean number of recovered signatures was 3,031.2, corresponding to a mean recovery rate of 60.00%.

The difference between the nominal TRIM-enabled and nominal TRIM-disabled results is therefore substantial on the NVMe SSD. The two deleted datasets under the TRIM-enabled condition produced a combined mean recovery rate of 0%, whereas the two deleted datasets under the TRIM-disabled condition produced a simple average recovery rate of 69.80%. This indicates that deleted data remained substantially more recoverable when the explicit TRIM operation was not performed.

The NVMe results also show considerable variability in the TRIM-disabled condition. DS10 had a standard deviation of approximately 44.50 percentage points, while DS12 had a standard deviation of approximately 54.77 percentage points. The high variation resulted from the contrast between near-complete recovery in some runs and no recovery in other runs. Therefore, although the nominal TRIM-disabled condition allowed substantial recovery, the outcome was not identical across repeated experiments. This variability is particularly visible in DS12. The dataset was recovered almost completely in the first three runs, but no signatures were recovered in the final two runs. This indicates that the presence of deleted data was not sufficient by itself to guarantee successful recovery. The

recoverability of the data changed between runs, suggesting that the state of the SSD, the storage controller, or the connection path influenced the final result.

SATA SSD Results

Table 4.2: Data Recovery Results After TRIM on SATA SSD

Dataset	Signatures Recovered				
	Run 1	**Run 2**	**Run 3**	**Run 4**	**Run 5**
DS1	5,052	5,052	5,052	5,070	5,052
DS2	56	19	0	0	0
DS3	5,052	5,052	5,052	5,052	5,052
DS4	47	37	0	0	0
DS5	5,052	5,052	5,052	5,052	5,052
DS6	5,052	5,052	5,052	5,052	5,052
DS7	5,052	5,052	5,052	5,052	5,103
DS8	5,052	5,052	5,052	5,052	5,103
DS9	5,052	5,052	5,052	5,052	5,052
DS10	56	0	0	0	0
DS11	5,052	5,052	5,052	5,052	5,052
DS12	72	4,487	0	0	0
DS13	5,052	5,052	5,052	5,052	5,052
DS14	5,052	5,052	5,052	5,052	5,052
DS15	5,052	5,052	5,052	5,052	5,052
DS16	5,052	5,052	5,052	5,052	5,052

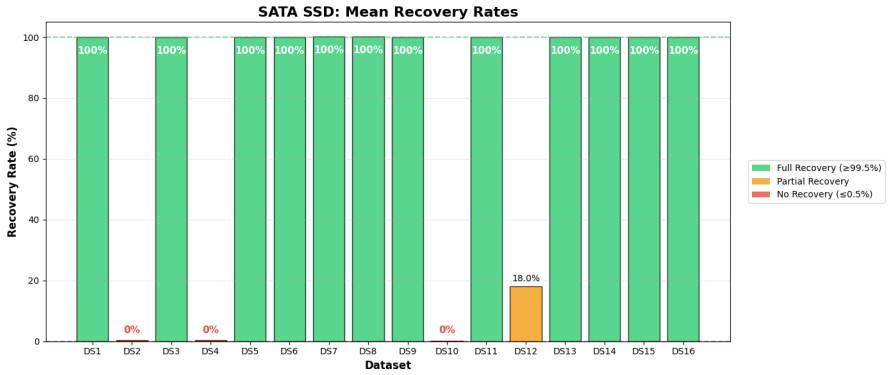


Figure 4.2: Mean Recovery on SATA SSD

The SATA SSD results showed a similar pattern for the control datasets. DS3, DS5, DS9, DS11, and DS13 produced 5,052 signatures in all five runs. DS1 produced 5,052 signatures in four runs and 5,070 signatures in one run. These results correspond to approximately 100% recovery and confirm that the recovery procedure was capable of identifying the expected signatures on the SATA device.

The post-deletion write datasets also showed almost complete recovery. DS6 produced 5,052 signatures in all five runs, while DS7 and DS8 produced 5,103 signatures in the fifth run and 5,052 signatures in the other four runs. DS14, DS15, and DS16 produced 5,052 signatures in every run. The mean recovery rates for these datasets were therefore approximately 100%. The values slightly above 100% resulted from the detection of more signature occurrences than the nominal expected value of 5,052 which were due to file remnants from previous runs being recovered by Photorec.

The deleted datasets under the nominal TRIM-enabled condition produced only a small number of signatures in the first two runs and no signatures in the remaining three runs. DS2 produced 56, 19, 0, 0, and 0 signatures, resulting in a mean of 15 recovered signatures and a mean recovery rate of approximately 0.30%. DS4 produced 47, 37, 0, 0, and 0 signatures, resulting in a mean of 16.8 recovered signatures and a mean recovery rate of approximately 0.33%.

These results indicate that the nominal TRIM-enabled condition was highly effective at preventing recovery of the deleted signatures from the SATA SSD. However, the results should not be described as absolute zero recovery because small numbers of signatures were detected in the first two runs for both DS2 and DS4. A more accurate interpretation is that recovery was reduced to a near-zero level, with the remaining signatures appearing only during the initial repetitions.

The nominal TRIM-disabled SATA results were more variable than the TRIM-enabled results, but the overall recovery level remained substantially lower than that observed for the NVMe SSD. DS10 produced 56 signatures in the first run and zero signatures in the remaining four runs. This produced a mean recovery rate of only 0.22%. DS12 produced 72 signatures in the first run, 4,487 signatures in the second run, and zero signatures in the final three runs. Its mean recovery rate was 18.05%.

DS12 was the only SATA dataset to show a substantial recovery event under the nominal TRIM-disabled condition. The second run recovered 4,487 of the expected 5,052 signatures, which corresponds to approximately 88.82% recovery in that individual run. However, because no signatures were recovered in the final three runs, the mean recovery rate across all five runs fell to 18.05%. This demonstrates why the individual run results are important. Reporting only the mean would conceal the large difference between the second run and the other four runs.

The standard deviation for DS12 was approximately 39.57 percentage points, which reflects this inconsistent recovery pattern. By comparison, the standard

deviation for DS10 was only approximately 0.50 percentage points because the dataset produced a small number of signatures in only the first run and zero signatures thereafter.

Comparison between the NVMe and SATA SSDs

The comparison shows that the NVMe SSD retained considerably more recoverable data under the nominal TRIM-disabled condition than the SATA SSD. The NVMe deleted datasets produced mean recovery rates of 79.60% and 60.00%, whereas the corresponding SATA datasets produced mean recovery rates of 0.22% and 18.05%. Taking the simple average of the two deleted datasets, the nominal TRIM-disabled recovery rate was 69.80% for the NVMe SSD and approximately 9.14% for the SATA SSD.

Under the nominal TRIM-enabled condition, both devices produced very low recovery rates. The NVMe SSD produced no recovered signatures for either deleted dataset. The SATA SSD produced less than 1% mean recovery for both deleted datasets. This indicates that the explicit TRIM operation was associated with a strong reduction in host-side recovery on both devices.

The results also demonstrate that TRIM behaviour was not uniform across the two SSDs. If the effect of TRIM were determined only by the operating-system deletion command, the two devices would be expected to show similar recovery patterns. Instead, the NVMe SSD showed substantial recovery when TRIM was disabled, while the SATA SSD showed limited recovery under the same nominal condition. This difference indicates that storage-device implementation and controller behaviour affected the persistence and recoverability of deleted data.

The results further show that repeated runs were necessary to identify the variability of the recovery process. The NVMe device produced large changes in recovery between runs for DS10 and DS12. The SATA device showed a similar pattern for DS12, where one run produced almost complete recovery while the other runs produced either minimal or no recovery. These results suggest that recovery should not be assessed using a single experimental run because a single run may either overestimate or underestimate the recoverability of deleted data.

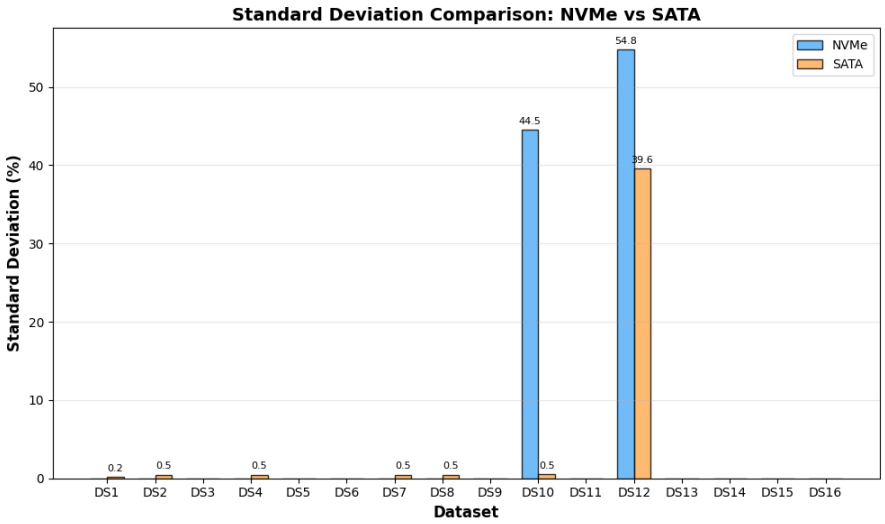


Figure 4.3: Standard Deviation of Recovery on NVMe and SATA SSDs

Interpretation of the recovery results

The results provide evidence that the nominal TRIM-enabled condition substantially reduced the recovery of deleted dataset signatures on both storage devices. The reduction was complete for the NVMe deleted datasets and almost complete for the SATA deleted datasets. In contrast, disabling TRIM allowed substantial recovery on the NVMe SSD and limited, but occasionally significant, recovery on the SATA SSD.

The findings therefore support the conclusion that TRIM configuration influenced host-side data recoverability. However, the results do not support the broader conclusion that TRIM always produces the same level of data destruction on every SSD. The difference between the NVMe and SATA results shows that the effectiveness of the operation depended on the storage device and its implementation.

The recovery results represent the output of the selected forensic procedure rather than a direct measurement of the physical condition of the NAND flash cells. A zero recovery result means that the selected recovery process did not identify the dataset-specific signatures. It does not independently prove that every physical copy of the data had been destroyed.

The connection path must also be considered when interpreting the comparison. The NVMe SSD was accessed through a USB enclosure, while the SATA SSD was accessed through a USB-to-SATA adapter. The host system exposed a discard-capable path, but this did not independently confirm that every discard request was forwarded to and processed by the SSD controller. Therefore, the results describe the behaviour of the complete tested configuration, including the operating system,

filesystem, drivers, USB connection, adapter or enclosure, SSD controller, and recovery procedure.

Overall, Experiment 2 demonstrates that deleted data was least recoverable when the nominal TRIM-enabled procedure was used. When TRIM was disabled, the NVMe SSD retained a substantial amount of recoverable data, while the SATA SSD showed a lower but inconsistent level of recovery. The difference between the devices confirms that secure deletion performance cannot be inferred from the deletion command alone and must be evaluated using repeated, device-specific recovery tests.

4.1.3 Experiment 3:Testing Recoverability at different fill levels and times intervals

The recovery-time experiment produced a consistent recovery result across all tested filesystem utilisation levels and recovery intervals. No files were recovered in any of the experimental conditions. At 25%, 75%, and 90% filesystem utilisation, the number of recovered files remained zero for immediate recovery and for recovery attempts performed after 5, 15, and 30 minutes. This result was consistent across both experimental runs. Therefore, within the 30-minute observation period, no measurable change in host-side recoverability was observed as a function of the time elapsed following the TRIM operation. In particular, the absence of recovered files at the immediate recovery point indicates that the data was already not recoverable through the selected forensic recovery procedure immediately following TRIM. The subsequent 5-, 15-, and 30-minute waiting periods did not produce any additional reduction in the measured recovery outcome because the recovery count was already zero at the initial measurement point.

Table 4.3: Recovery Results

Fill Level	Recovery Delay	SATA SSD Recovered	NVMe SSD Recovered
25%	0 min	0	0
25%	5 min	0	0
25%	15 min	0	0
25%	30 min	0	0
75%	0 min	0	0
75%	5 min	0	0
75%	15 min	0	0
75%	30 min	0	0
90%	0 min	0	0
90%	5 min	0	0
90%	15 min	0	0
90%	30 min	0	0

The measured TRIM execution times showed a different pattern. The first fstrim operation following deletion generally required substantially more time than subsequent operations. In the first experimental run, the real execution time for the initial TRIM operation was 4.53 seconds at 25% utilisation, 7.20 seconds at 75% utilisation, and 7.80 seconds at 90% utilisation. The corresponding subsequent measurements were generally below approximately one second. A similar pattern was observed in the second experimental run. The initial TRIM operations required 8.36 seconds, 32.30 seconds, and 16.15 seconds at 25%, 75%, and 90% utilisation, respectively. Subsequent TRIM operations generally completed within approximately one to three seconds. The longer execution time observed for the first TRIM operation is consistent with the larger amount of logical storage being discarded after deletion of the experimental dataset. The amount reported as trimmed during the first operation was substantially larger than that reported during the subsequent measurements. However, the variation between runs, particularly at the 75% and 90% utilisation levels, indicates that TRIM execution time was not determined solely by filesystem utilisation. Controller-level storage management and the state of the device may also influence the host-observed execution time.

Table 4.4: Recovery Time Results

Fill Level	Drive	0 min	5 min	15 min	30 min
25%	SATA	4.53 s	0.62 s	0.87 s	0.73 s
25%	NVMe	8.36 s	0.39 s	0.40 s	1.14 s
75%	SATA	7.20 s	0.72 s	0.66 s	0.73 s
75%	NVMe	32.30 s	2.87 s	1.26 s	1.18 s
90%	SATA	7.80 s	1.01 s	1.00 s	1.00 s
90%	NVMe	16.15 s	0.96 s	1.20 s	1.19 s

Overall, the results indicate that, under the tested conditions, the elapsed time between TRIM and forensic recovery did not produce an observable change in recoverability during the 30-minute observation period. The results also show that the host-visible execution time of TRIM varied according to the amount of data being discarded and the state of the storage device.

4.1.4 Experiment 4: Cryptographic Erase Attempt

Table 4.5: Results of ATA Sanitization Command Attempts

Command	Output	Interpretation
`hdparm -security-erase`	`SECURITY_SET_PASS: Inappropriate ioctl for device`	Command rejected by kernel/bridge
`hdparm -sanitize-crypto-scramble`	No output	Command not forwarded or acknowledged
`hdparm -sanitize-block-erase`	No output	Command not forwarded or acknowledged
`hdparm -sanitize-status`	No output	Status query not forwarded

None of the cryptographic erase commands produced a confirmation message or a status report. The `-security-erase` command failed explicitly with `Inappropriate ioctl for device`, indicating that the kernel or the USB bridge rejected the ATA Security command before it reached the drive. The `-sanitize-*` commands produced no output at all, which is a different failure mode: the

command was accepted by the command-line tool but produced no confirmation that it was received or processed by the drive.

Because the drive's identify data confirmed support for both the `Security Mode` and `SANITIZE` feature sets, the failure cannot be attributed to a lack of drive capability. The most plausible explanation is that the USB-to-SATA bridge did not forward these commands to the drive. This is consistent with the earlier observation that the bridge required explicit udev configuration to forward TRIM/UNMAP commands, and it extends that finding to a broader class of drive-level sanitization commands.

The experiment therefore produced a negative result: cryptographic erase could not be tested through the USB bridge. However, this negative result is itself a finding, because it demonstrates that the connection path can silently block drive-level sanitization commands. The absence of an error message for the `SANITIZE` commands is particularly significant: a user might reasonably interpret the lack of output as a successful operation when in fact no sanitization occurred.

Chapter 5

Discussion of Findings

This chapter discusses the findings obtained from the experimental evaluation of secure deletion techniques in modern storage environments. The discussion relates the experimental observations to the research questions concerning data recoverability, TRIM behaviour, execution time, filesystem mechanisms, storage-device variation, and the limitations of host-side verification.

The central finding is that secure deletion cannot be evaluated reliably by observing only the successful completion of a deletion command. The experiments demonstrate that the outcome observed by the host depends on the interaction between the filesystem, the storage interface, the controller, the Flash Translation Layer, and the forensic recovery procedure. Therefore, the results should be interpreted in terms of observed host-side recoverability, rather than as direct proof of the physical state of the NAND cells.

5.1 Summary of Main Findings

Experiment 1

The first experiment demonstrated that deletion changed the ext4 filesystem metadata significantly. The deleted inode had a size of zero, a block count of zero, no remaining links, and an empty extent structure. These observations confirm that the file was logically unlinked and that the filesystem no longer maintained usable block references to the deleted file. The journal analysis showed that deletion-related metadata was recorded in the filesystem journal, but the journal did not contain the actual file data. The recovery attempt using ext4magic was unsuccessful, and direct inspection of the previously allocated blocks showed only zero bytes. However, the zero values cannot by themselves prove that the NAND cells were physically erased. They may represent physical erasure, controller-level

zero return, or another abstraction at the storage interface. The stronger conclusion is that the deleted data was not recoverable through the host-side block-access procedure used in the experiment.

Experiment 2

Experiment 2 compared data recovery under TRIM-enabled and TRIM-disabled conditions on an NVMe SSD and a SATA SSD. The results demonstrated that the recovery outcome depended on both the TRIM condition and the storage device.

The positive-control datasets were recovered at or near the expected level on both SSDs. This confirms that the recovery procedure was capable of identifying the embedded dataset signatures when the data remained available. The recovery of the kept datasets also provides confidence that the zero or near-zero results observed for the deleted datasets were not caused simply by a complete failure of the recovery tool. The nominal TRIM-enabled condition produced the lowest recovery rates. On the NVMe SSD, both deleted datasets, DS2 and DS4, produced zero recovered signatures in all five runs. On the SATA SSD, DS2 and DS4 produced only a small number of signatures during the first two runs and no signatures in the remaining runs. Their mean recovery rates were approximately 0.30% and 0.33%, respectively.

The nominal TRIM-disabled condition produced a substantially different result on the NVMe SSD. DS10 produced a mean recovery rate of 79.60%, while DS12 produced a mean recovery rate of 60.00%. These values demonstrate that a considerable proportion of the deleted signatures remained observable through the host-side recovery procedure when the explicit TRIM operation was not performed.

The SATA SSD showed lower recovery under the nominal TRIM-disabled condition. DS10 produced a mean recovery rate of only 0.22%, while DS12 produced a mean recovery rate of 18.05%. The result for DS12 was strongly affected by one run in which 4,487 of the expected 5,052 signatures were recovered. The remaining four runs produced either 72 signatures or no signatures. This shows that the mean recovery rate alone does not fully describe the behaviour of the device. Individual run results are necessary to identify the considerable variation between repetitions.

The results indicate that TRIM was associated with a strong reduction in host-side recovery on both devices. However, the effect was not identical across the NVMe and SATA SSDs. The NVMe SSD retained substantially more recoverable data under the nominal TRIM-disabled condition, whereas the SATA SSD showed limited and inconsistent recovery. This difference suggests that the storage controller, firmware, device state, and connection path influenced the persistence of the deleted data.

The standard deviation results support this interpretation. The NVMe results showed standard deviations of approximately 44.50 percentage points for DS10 and

54.77 percentage points for DS12. The SATA results showed a standard deviation of approximately 39.57 percentage points for DS12. These high values were caused by large differences between repeated runs, including runs with almost complete recovery and runs with no recovery.

Experiment 2 therefore does not support the assumption that a TRIM-disabled SSD will always retain the same amount of recoverable data. Instead, the results demonstrate that recoverability is variable and device-specific. The nominal TRIM condition provided a useful experimental distinction, but the observed results cannot be attributed exclusively to the operating-system setting because the two SSDs used different physical devices and different USB connection paths.

Experiment 4: Cryptographic Erase Attempt

The cryptographic erase experiment produced a negative result that is itself informative. Neither the `ATA Security Erase` command nor the `ATA SANITIZE` commands could be executed through the USB-to-SATA bridge. The `-security-erase` command failed with `Inappropriate ioctl for device`, which is the standard kernel response when an ioctl is not supported by the underlying device or driver. The `-sanitize-*` commands produced no output at all, suggesting that the bridge did not forward them and did not return an error.

This finding extends the connection-path limitation identified in Experiment 2. In that experiment, the USB-NVMe enclosure required explicit udev configuration before TRIM/UNMAP commands were forwarded. The SATA SSD, accessed through a different USB-to-SATA adapter, showed inconsistent TRIM behaviour. Experiment 4 shows that the same class of limitation applies to `ATA Security` and `SANITIZE` commands: these commands are not reliably forwarded by USB bridges.

The implication is that drive-level sanitization commands cannot be assumed to execute correctly when a USB bridge is present in the connection path. Organisations attempting to sanitize SSDs through USB adapters may believe the operation succeeded when in fact the command was never delivered to the drive. This is a practical verification failure that compounds the broader verifiability crisis discussed in Section 2.3.

The experiment also illustrates a distinction between two failure modes. The `-security-erase` command produced an explicit error, which at least informed the user that the operation had not been performed. The `-sanitize-*` commands produced no output, leaving the user with no indication of success or failure. The second mode is more dangerous because it can be mistaken for success. A more robust sanitization procedure would require confirmation from the drive itself, not merely the absence of an error message from the host-side tool.

Legal and Compliance Implications

The experimental results show why the completion status returned by a storage device cannot automatically be treated as conclusive proof of secure deletion. The thesis identifies a verification gap across several deletion techniques and notes that cryptographic erasure, controller-level commands, and other methods may provide limited visibility into the physical state of the medium. This does not make accountability impossible. It means that a controller must combine technical evidence with governance evidence and must avoid making claims stronger than the evidence supports.

The experiments can therefore support legal compliance in three ways. First, they can help determine whether a deletion method is suitable for a particular storage technology. Second, they can identify conditions under which a successful command may not correspond to complete sanitisation. Third, they can define the limitations that must be disclosed and managed when the controller documents the deletion process.

The appropriate compliance statement is consequently not that a controller has proved the physical destruction of every copy in every possible state. It is that the controller used a method appropriate to the medium and risk, executed it under controlled conditions, verified the result to the extent technically possible, addressed known copies and exceptions, and retained sufficient evidence to demonstrate that the erasure obligation was handled responsibly.

This distinction also explains why the absence of an independent verification method is itself a result with legal relevance. Where the technical evidence is weak, the controller may need to use a stronger sanitisation method, such as cryptographic erasure with reliable key-management evidence, device-level sanitisation, or physical destruction. A controller should not rely solely on a logical deletion event or an unverified vendor success signal where the consequences of residual data exposure are significant.

5.2 Limitations of the Study

The first limitation concerns the number and diversity of storage devices. The experiments used two approximately 128 GB SSDs, one NVMe device and one SATA device. The results therefore cannot be generalised to all SSDs, manufacturers, firmware versions, capacities, or interface implementations.

A significant limitation of the study is the use of USB bridge hardware between the host system and both SSDs. The NVMe SSD was connected through a JMicron JMS583 USB enclosure, while the SATA SSD was connected through a USB-to-SATA adapter. These devices introduced additional firmware and command-translation layers into the storage path. The NVMe enclosure initially did not

expose TRIM support to Ubuntu. Discard or unmap behaviour had to be explicitly enabled before the host reported discard capability. This demonstrates that the presence of a physical connection between the host and SSD does not guarantee correct forwarding of discard commands. The bridge may have affected both the recovery results and the measured execution times. For example, the 0% recovery observed for the TRIM-enabled NVMe condition may be associated with successful command forwarding, controller-level masking, or physical processing by the SSD. The low and similar recovery rates observed on the SATA SSD may likewise have been influenced by the SATA bridge or by the way it handled TRIM-related requests. The experiment cannot determine which of these explanations is correct because commands were not captured at the bridge-to-device boundary and the NAND state was not directly examined. Therefore, the bridge should be treated as a potential confounding variable rather than as a confirmed cause of the results. This limitation also affects comparisons between the two SSDs. The observed differences cannot be attributed exclusively to NVMe and SATA interfaces because the devices were accessed through different connection paths. The findings are valid for the tested configurations, but they should not be generalised without additional experiments using direct native SATA and PCIe connections.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

6.2 Future Work

Future research should repeat the experiments using direct native connections. The NVMe SSD should be connected directly through a PCIe or M.2 interface, while the SATA SSD should be connected directly to a native SATA port without a USB-to-SATA adapter. The same dataset, filesystem, deletion procedure, TRIM conditions, recovery tools, fill levels, and recovery delays should then be applied to the directly connected devices. his design would help distinguish the effect of the SSD controller from the effect of the bridge. It would also allow the same SSD to be tested through different connection paths, reducing the risk that device model differences are mistaken for interface effects. Future work should include more SSD models, different bridge chipsets, direct and USB-connected configurations, multiple filesystems, and longer observation periods. Recovery should also be attempted using more than one forensic method, including specialised laboratory techniques where available. These changes would improve the external validity of the framework and help determine whether atypical recovery results arise from SSD implementation, bridge behaviour, filesystem configuration, or the limits of the recovery procedure.

Future work should also evaluate cryptographic erase and block erase sanitization commands using direct native connections. The present study was unable to test these commands because the USB bridge did not forward them to the drive. Repeating the experiment with the SATA SSD connected directly to a native SATA port would allow the effectiveness and latency of ATA Security Erase, SANITIZE

Crypto Scramble, and SANITIZE Block Erase to be measured and compared against TRIM and overwriting. The same applies to the NVMe SSD, which should be tested through a direct M.2 or PCIe connection using the `nvme sanitize` command. These experiments would complete the comparison matrix by adding measured results for the drive-level sanitization techniques that are currently discussed only theoretically in Chapter 2.

Appendix A

Appendix A

A.1 Device Specifications

A.1.1 Host system

```
hostnamectl
Static hostname: anji-Nitro-AN515-58
Icon name: computer-laptop
Chassis: laptop
Machine ID: 07bcf4e12b18401998b1ced41087efdf
Boot ID: df0e8cb23e204d1080c42530bfeb49ad
Operating System: Ubuntu 26.04 LTS

Kernel: Linux 7.0.0-28-generic
Architecture: x86-64
Hardware Vendor: Acer
Hardware Model: Nitro AN515-58
Hardware SKU: 0000000000000000
Hardware Version: V2.21
Firmware Version: V2.21
Firmware Date: Tue 2026-06-09
Firmware Age: 1month 4w
```

A.1.2 Test Drive 1: NVMe SSD

```
sudo smartctl -a /dev/sda

smartctl 7.5 2025-04-30 r5714[x86_64-linux-7.0.0-28-generic](local build)
Copyright(C)2002-25, Bruce Allen, Christian Franke,www.smartmontools.org
```

```
=== START OF INFORMATION SECTION ===
Model Number:                       PSELN128GA87MC0
Serial Number:                      0524115009487
Firmware Version:                   HPE4BP01
PCI Vendor/Subsystem ID:            0x1987
IEEE OUI Identifier:                0x6479a7
Controller ID:                      1
NVMe Version:                       1.4
Number of Namespaces:               1
Namespace 1 Size/Capacity:          128,035,676,160 [128 GB]
Namespace 1 Formatted LBA Size:     512
Namespace 1 IEEE EUI-64:            6479a7 8d8f00250f
Local Time is:                      Thu Aug  6 12:04:21 2026 CEST
Firmware Updates (0x04):            2 Slots
Optional Admin Commands (0x0017):   Security Format Frmw_DL Self_Test
Optional NVM Commands (0x0056):Wr_Unc DS_Mngmt Sav/Sel_Feat Timestmp
Log Page Attributes (0x1e):Cmd_Eff_Lg Ext_Get_Lg Telmtry_Lg Pers_Ev_Lg
Maximum Data Transfer Size:64 Pages
Warning  Comp. Temp. Threshold:     83 Celsius
Critical Comp. Temp. Threshold:     85 Celsius

Supported Power States
St Op     Max   Active     Idle   RL RT WL WT  Ent_Lat  Ex_Lat
0 +     4.30W       -        -    0  0  0  0        0       0
1 +     3.00W       -        -    1  1  1  1        0       0
2 +     1.50W       -        -    2  2  2  2        0       0
3 -   0.1000W       -        -    3  3  3  3     5000   10000
4 -   0.0025W       -        -    4  4  4  4     5000   45000

Supported LBA Sizes (NSID 0x1)
Id Fmt  Data  Metadt  Rel_Perf
0 +     512       0         1
1 -    4096       0         0

=== START OF SMART DATA SECTION ===
SMART overall-health self-assessment test result: PASSED

SMART/Health Information (NVMe Log 0x02, NSID 0xffffffff)
Critical Warning:                   0x00
Temperature:                        57 Celsius
```

```
Available Spare:                    100%
Available Spare Threshold:          5%
Percentage Used:                    0%
Data Units Read:                    1,417,462 [725 GB]
Data Units Written:                 393,172 [201 GB]
Host Read Commands:                 11,854,846
Host Write Commands:                2,109,150
Controller Busy Time:               36
Power Cycles:                       72
Power On Hours:                     11
Unsafe Shutdowns:                   24
Media and Data Integrity Errors:    0
Error Information Log Entries:      64
Warning  Comp. Temperature Time:    0
Critical Comp. Temperature Time:    0
Temperature Sensor 1:               57 Celsius
```

A.1.3 Test Drive 2: SATA SSD

```
sudo hdparm -I /dev/sdb

/dev/sdb:

ATA device, with non-removable media
Model Number:       EDILOCA  ES106 128GB

Serial Number:      26060047623000204

Firmware Revision:  HC2705H3
Media Serial Num:

Media Manufacturer:
Transport:Serial, ATA8-AST,SATA 1.0a,SATA II Extensions,SATA Rev 2.5,SATA Rev 2
Standards:
Used: unknown (minor revision code 0x0110)
Supported: 9 8 7 6 5
Likely used: 9
Configuration:
Logical  max current
cylinders 16383 16383
heads  16 16
```

```
sectors/track 63 63
--
CHS current addressable sectors:    16514064
LBA    user addressable sectors:   250069680
LBA48  user addressable sectors:   250069680
Logical  Sector size:                   512 bytes
Physical Sector size:                   512 bytes
Logical Sector-0 offset:                  0 bytes
device size with M = 1024*1024:      122104 MBytes
device size with M = 1000*1000:      128035 MBytes (128 GB)
cache/buffer size  = unknown
Form Factor: 2.5 inch
Nominal Media Rotation Rate: Solid State Device
Capabilities:
LBA, IORDY(can be disabled)
Queue depth: 32
Standby timer values: spec'd by Standard, no device specific minimum
R/W multiple sector transfer: Max = 8 Current = 1
DMA: mdma0 mdma1 mdma2 udma0 udma1 udma2 udma3 udma4 udma5 *udma6
Cycle time: min=120ns recommended=120ns
PIO: pio0 pio1 pio2 pio3 pio4
Cycle time: no flow control=120ns  IORDY flow control=120ns
Commands/features:
Enabled Supported:
* SMART feature set
Security Mode feature set
* Power Management feature set
* Write cache
* Look-ahead
* WRITE_BUFFER command
* READ_BUFFER command
* NOP cmd
* 48-bit Address feature set
* Mandatory FLUSH_CACHE
* FLUSH_CACHE_EXT
* SMART error logging
* SMART self-test
* General Purpose Logging feature set
* WRITE_{DMA|MULTIPLE}_FUA_EXT
* 64-bit World wide name
* {READ,WRITE}_DMA_EXT_GPL commands
```

```
* Gen1 signaling speed (1.5Gb/s)
* Gen2 signaling speed (3.0Gb/s)
* Gen3 signaling speed (6.0Gb/s)
* Native Command Queueing (NCQ)
* Phy event counters
* Host automatic Partial to Slumber transitions
* Device automatic Partial to Slumber transitions
* READ_LOG_DMA_EXT equivalent to READ_LOG_EXT
DMA Setup Auto-Activate optimization
* Software settings preservation
* SANITIZE feature set
* BLOCK_ERASE_EXT command
* WRITE BUFFER DMA command
* READ BUFFER DMA command
* Data Set Management TRIM supported (limit 8 blocks)
Security:
Master password revision code = 65534
supported
not enabled
not locked
not frozen
not expired: security count
supported: enhanced erase
2min for SECURITY ERASE UNIT. 2min for ENHANCED SECURITY ERASE UNIT.
Logical Unit WWN Device Identifier:0000000000000000
NAA  : 0
IEEE OUI : 000000
Unique ID : 000000000
Checksum: correct
```

A.2 Experiment 1

```
sudo mkdir -p /mnt/test/test_data
sudo openssl rand -out "/mnt/test/test_data/
inode_test001.dat" 104857600


sudo find "/mnt/test/test_data" -type f -exec stat -c '%n
| Inode: %i' {} \;
/mnt/test/test_data/inode_test001.dat | Inode: 6291458
```

```
sudo dumpe2fs -h /dev/sda1 | grep -E "Journal|Inode"

sudo rm -rf "/mnt/test/test_data"/*

sudo debugfs -R "stat <6291458>" /dev/sda1

sudo dumpe2fs -h /dev/sda1 | grep -E "Journal|Inode"

sudo dumpe2fs -h /dev/sda1 | grep -E "Journal|Inode"

sudo ext4magic /dev/sda1 -j /dev/sda1 -I 6291458 -d /tmp/recovery/

sudo ext4magic /dev/sda1 -j /dev/sda1 -I 6291458 -B 620 -d
/tmp/recovery/

sudo debugfs /dev/sda1
debugfs: ncheck 6291458

sudo debugfs -R "imap <6291458>" /dev/sda1

sudo debugfs -R "logdump" /dev/sda1
sudo debugfs -R "logdump -a" /dev/sda1

sudo dd if=/dev/sda1 bs=4096 skip=33388 count=1 2>/dev/null | xxd
| head -20
sudo dd if=/dev/sda1 bs=4096 skip=33389 count=1 2>/dev/null | xxd
| head -20

sudo dd if=/dev/sda1 of=~/inode_analysis/journal_tx61.bin bs=4096
skip=15237743 count=1
sudo dd if=~/inode_analysis/journal_tx61.bin
of=~/inode_analysis/inode_tx61.bin bs=1 skip=256 count=256
```

A.3 Experiment 2

A.3.1 Experimental Setup

```
 # 1. Unmount if mounted
sudo umount /dev/sda1 2>/dev/null
```

```
# 2. Wipe the drive (quick wipe)
sudo dd if=/dev/zero of=/dev/sda bs=1M count=100 status=progress

# 3. Create GPT partition table
sudo parted /dev/sda mklabel gpt

# 4. Create single ext4 partition
sudo parted /dev/sda mkpart primary ext4 0% 100%

# 5. Reload partition table
sudo partprobe /dev/sda
sleep 2

# 6. Format as ext4
sudo mkfs.ext4 -F /dev/sda1

# 7. Create mount point
sudo mkdir -p /mnt/{MountPoint}
```

A.3.2 Trim-on and Trim-off Conditions

```
# TRIM ON
sudo mount -o discard /dev/sda1 /mnt/{MountPoint}

# TRIM OFF
sudo mount -o nodiscard /dev/sda1 /mnt/{MountPoint}
```

A.3.3 Data Generation Script

```
1
2  #!/bin/bash
3  # Data generation script
4
5  MOUNT_POINT=$1
6  DATASET_NUM=$2
7  UNIQUE_ID=$(openssl rand -hex 8)
8
9  FOLDER_NAME="TEST_DATA_${DATASET_NUM}_${UNIQUE_ID}"
10 mkdir -p "${MOUNT_POINT}/${FOLDER_NAME}"
11
12 # File 1: Small text file (51 signatures)
```

```
13 cat > "${MOUNT_POINT}/${FOLDER_NAME}/small.txt" << ENDTEXT
14 UNIQUE_SIGNATURE_DS${DATASET_NUM}_SMALL
15 $(for i in {1..50}; do echo "UNIQUE_SIGNATURE_DS${DATASET_NUM}
       _SMALL"; done)
16 ENDTEXT
17
18 # File 2: Medium text file (5,001 signatures)
19 cat > "${MOUNT_POINT}/${FOLDER_NAME}/medium.txt" << ENDTEXT
20 UNIQUE_SIGNATURE_DS${DATASET_NUM}_MEDIUM
21 $(for i in {1..5000}; do echo "UNIQUE_SIGNATURE_DS${DATASET_NUM}
       _MEDIUM"; done)
22 ENDTEXT
23
24 # File 3: Large binary file (10MB + 1 signature)
25 dd if=/dev/urandom of="${MOUNT_POINT}/${FOLDER_NAME}/large.bin" bs
       =1M count=10 status=none
26 echo "UNIQUE_SIGNATURE_DS${DATASET_NUM}_LARGE" >> "${MOUNT_POINT}/
       ${FOLDER_NAME}/large.bin"
27
28 # File 4: 500 small text files (500 signatures)
29 for i in {1..500}; do
30     echo "UNIQUE_SIGNATURE_DS${DATASET_NUM}_SMALL_${i}" > "${
       MOUNT_POINT}/${FOLDER_NAME}/tiny_${i}.txt"
31 done
32
33 # File 5: Binary image file (no signatures)
34 dd if=/dev/urandom of="${MOUNT_POINT}/${FOLDER_NAME}/image.jpg" bs
       =1K count=50 status=none
35
36 # File 6: Binary PDF file (no signatures)
37 dd if=/dev/urandom of="${MOUNT_POINT}/${FOLDER_NAME}/document.pdf"
       bs=1K count=30 status=none
38
39 # Create checksum file for verification
40 md5sum "${MOUNT_POINT}/${FOLDER_NAME}"/* > "${MOUNT_POINT}/${
       FOLDER_NAME}/checksums_${DATASET_NUM}.md5"
41
42 echo "Created dataset $DATASET_NUM in ${MOUNT_POINT}/${FOLDER_NAME
       }"
```

Listing A.1: Data Generation Script

A.3.4 Command List

Phase A: TRIM ON Procedure

1. Mount with TRIM enabled:

```
sudo mount -o discard /dev/sda1 /mnt/ssd_test
```

2. Create Datasets 1-5 with unique signatures:

```
for i in {1..5}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

3. Delete Datasets 2 and 4:

```
sudo rm -rf /mnt/ssd_test/TEST_DATA_2_*
sudo rm -rf /mnt/ssd_test/TEST_DATA_4_*
```

4. Force TRIM:

```
sudo fstrim -v /mnt/ssd_test
```

5. Create Overwrite Datasets 6-8:

```
for i in {6..8}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

6. Unmount the drive:

```
sudo umount /mnt/ssd_test
```

Phase B: TRIM OFF Procedure

1. Mount with TRIM enabled:

```
sudo mount -o discard /dev/sda1 /mnt/ssd_test
```

2. Create Datasets 1-5 with unique signatures:

```
for i in {1..5}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

3. Delete Datasets 2 and 4:

```
sudo rm -rf /mnt/ssd_test/TEST_DATA_2_*
sudo rm -rf /mnt/ssd_test/TEST_DATA_4_*
```

4. Force TRIM:

```
sudo fstrim -v /mnt/ssd_test
```

5. Create Overwrite Datasets 6-8:

```
for i in {6..8}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

6. Unmount the drive:

```
sudo umount /mnt/ssd_test
```

Phase B: TRIM OFF Procedure

1. Mount without TRIM:

```
sudo mount -o nodiscard /dev/sda1 /mnt/ssd_test
```

2. Create Datasets 9-13:

```
for i in {9..13}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

3. Delete Datasets 10 and 12:

```
sudo rm -rf /mnt/ssd_test/TEST_DATA_10_*
sudo rm -rf /mnt/ssd_test/TEST_DATA_12_*
```

4. NO TRIM - deliberately skipped

5. Create Overwrite Datasets 14-16:

```
for i in {14..16}; do
    ./create_dataset.sh /mnt/ssd_test $i
done
```

6. Unmount the drive:

```
sudo umount /mnt/ssd_test
```

```
for i in {1..16}; do
count=$(grep -r "UNIQUE_SIGNATURE_DS${i}_". 2>/dev/null | wc -l)
echo "Dataset $i: $count files found"
done
```

Limitations and workarounds:

1. Identify the USB enclosure's chip and obtain its vendor and product ID using the command:

```
lsusb
```

2. Created udev rule to enable unmap

```
sudo nano /etc/udev/rules.d/90-usb-discard.rules
echo 'ACTION=="add|change", ATTRS{idVendor}=="152d",
ATTRS{idProduct}=="a583", SUBSYSTEM=="scsi_disk",
ATTR{provisioning_mode}="unmap"' | sudo tee
/etc/udev/rules.d/90-usb-discard.rules
```

3. Reloaded udev rules and replugged the device

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

4. Verified TRIM was recognized. `DISC-GRAN and DISC-MAX` values should be non-zero.

```
lsblk --discard
# DISC-GRAN=4K, DISC-MAX=4G (TRIM recognized)
```

A.4 Experiment 3: Full Script

```

# ============================================
# PHASE 1: wipe drive
# ============================================

TEST_DEVICE="/dev/sda"
MOUNT_POINT="/mnt/test_ssd"

echo ""
echo "=== PHASE 1: Wipe Drive ==="
echo "WARNING: This will erase ALL data on $TEST_DEVICE"
read -p "Type 'y' to continue: " confirm

if [ "$confirm" != "y" ]; then
    echo "Aborted."
    exit 1
fi


sudo umount $MOUNT_POINT 2>/dev/null
echo "Overwriting $TEST_DEVICE with zeros..."
sudo dd if=/dev/zero of=$TEST_DEVICE bs=1M count=100 status=
   progress

echo "Creating partition table..."
sudo parted -s $TEST_DEVICE mklabel gpt
sudo parted -s $TEST_DEVICE mkpart primary ext4 0% 100%
sudo mkfs.ext4 -F ${TEST_DEVICE}1

sudo mkdir -p $MOUNT_POINT
sudo mount ${TEST_DEVICE}1 $MOUNT_POINT
sudo chmod 777 $MOUNT_POINT
echo "Wipe complete"
echo "Trigger udev rules to enable trim"
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Listing A.2: PHASE 1: wipe drive and prepare for experiment

```
# ============================================
# SIMPLE CONFIGURATION
# ============================================

LOG_DIR="$HOME/trim_experiment/logs_nvme"
TEST_DIR="$MOUNT_POINT/test_data"
FILLER_DIR="$MOUNT_POINT/filler_data"

```

```
9  echo "Input Fill Level:"
10 read -p "Fill level (%): " FILL_LEVEL_INPUT
11
12 # test parameters
13 FILL_LEVEL=$FILL_LEVEL_INPUT
14 RECOVERY_WAITS=(0 5 15 30)
15 ITERATIONS=${#RECOVERY_WAITS[@]}
16
17 echo ""
18 echo "Configuration:"
19 echo "  Fill level: ${FILL_LEVEL}%"
20 echo "  Test data: ${TEST_DATA_SIZE_MB}MB"
21 echo "  Recovery waits: ${RECOVERY_WAITS[@]} minutes"
22 echo "  Total iterations: ${ITERATIONS}"
23 echo ""
24
25 mkdir -p $LOG_DIR
26 sudo mkdir -p $TEST_DIR $FILLER_DIR
27
28
29 # ============================================
30 # PHASE 2: Fill Drive to FILL_LEVEL%
31 # ============================================
32
33 echo ""
34 echo "=== PHASE 2: Fill Drive to ${FILL_LEVEL}% ==="
35
36 # Calculate how much to fill
37 TOTAL_KB=$(df --output=size "$MOUNT_POINT" | tail -1)
38 USED_KB=$(df --output=used "$MOUNT_POINT" | tail -1)
39 TARGET_KB=$((TOTAL_KB * FILL_LEVEL / 100))
40 FILL_KB=$((TARGET_KB - USED_KB))
41
42 echo "Need to write $((FILL_KB / 1024)) MB to reach ${FILL_LEVEL}%
     "
43
44 # Create filler files
45 written_kb=0
46 file_count=0
47 while [ $written_kb -lt $FILL_KB ]; do
48     file_name="$FILLER_DIR/filler_$(printf "%04d" $file_count).dat
     "
49     this_size=$((FILL_KB - written_kb))
50     if [ $this_size -gt $((1024 * 100)) ]; then  # 100MB chunks
51         this_size=$((1024 * 100))
52     fi
53
54     sudo dd if=/dev/urandom of="$file_name" bs=1K count=$this_size
       status=none
```

```
    written_kb=$((written_kb + this_size))
    file_count=$((file_count + 1))

    echo -ne "\r  Progress: $((written_kb * 100 / FILL_KB))%"
done
echo ""
echo "Created $file_count filler files"

# Check actual fill level
NEW_USED=$(df --output=used "$MOUNT_POINT" | tail -1)
NEW_PERCENT=$((NEW_USED * 100 / TOTAL_KB))
echo "Actual fill level: ${NEW_PERCENT}%"
```

Listing A.3: PHASE 2: Configure andFill Drive to Target Utilisation Level

```
# ============================================
# MAIN EXPERIMENT LOOP - Each iteration tests complete cycle
# ============================================

echo ""
echo "=== PHASE 3-6: MAIN EXPERIMENT ($ITERATIONS iterations) ==="
echo "Each iteration: Create -> Delete -> TRIM -> Recover"
echo ""

# Initialize results files
if [ ! -f "$LOG_DIR/experiment_results.csv" ]; then
    echo "run_timestamp,iteration,wait_minutes,fill_level,
    real_time,user_time,sys_time,trimmed_bytes,recovered_files" >
    $LOG_DIR/experiment_results.csv
fi

# Main loop - one iteration per recovery wait time
for ITER in $(seq 1 $ITERATIONS); do
    # Get the wait time for THIS iteration (0-indexed array)
    WAIT_INDEX=$((ITER - 1))
    CURRENT_WAIT=${RECOVERY_WAITS[$WAIT_INDEX]}

    echo "========================================"
    echo "ITERATION $ITER of $ITERATIONS - Wait: ${CURRENT_WAIT}
    minutes"
    echo "========================================"

# ============================================
# PHASE 3: Create Test Data
# ============================================

echo ""
echo "=== [ITER $ITER - PHASE 3]: Create Test Data ==="

```

```
32 # Create UNIQUE test data for THIS iteration
33 UNIQUE_ID=$(openssl rand -hex 8)
34 ITER_TEST_DIR="$TEST_DIR/TEST_DATA_${ITER}_${UNIQUE_ID}"
35 sudo mkdir -p "$ITER_TEST_DIR"
36
37 echo "Creating dataset in: $ITER_TEST_DIR"
38 echo ""
39
40 # ============================================
41 # FILE 1: Small text file
42 # ============================================
43
44 echo "Creating small text file..."
45
46 {
47     echo "UNIQUE_SIGNATURE_DS${ITER}_SMALL"
48     for i in {1..50}; do
49         echo "UNIQUE_SIGNATURE_DS${ITER}_SMALL"
50     done
51 } | sudo tee "$ITER_TEST_DIR/small.txt" > /dev/null
52
53 echo "small.txt created"
54
55
56 # ============================================
57 # FILE 2: Medium text file
58 # ============================================
59
60 echo "Creating medium text file..."
61
62 {
63     echo "UNIQUE_SIGNATURE_DS${ITER}_MEDIUM"
64     for i in {1..5000}; do
65         echo "UNIQUE_SIGNATURE_DS${ITER}_MEDIUM"
66     done
67 } | sudo tee "$ITER_TEST_DIR/medium.txt" > /dev/null
68 echo "medium.txt created"
69
70
71 # ============================================
72 # FILES 3-7: LARGE VALID ZIP FILES
73 # Approximately 1 GiB each
74 # ============================================
75
76 echo ""
77 echo "Creating large ZIP dataset (~2.5 GiB each)..."
78
79
80     TEMP_FILE="$ITER_TEST_DIR/.temporary_data.bin"
```

```
81      ZIP_FILE="$ITER_TEST_DIR/large_archive.zip"
82
83      sudo dd if=/dev/urandom \
84          of="$TEMP_FILE" \
85          bs=1M \
86          count=2560 \
87          status=progress
88
89
90      echo "UNIQUE_SIGNATURE_DS${ITER}_LARGE" | \
91          sudo tee -a "$TEMP_FILE" > /dev/null
92
93      sudo zip -0 -q "$ZIP_FILE" "$TEMP_FILE"
94
95      # Remove the temporary uncompressed file.
96      sudo rm -f "$TEMP_FILE"
97
98
99  echo ""
100 echo "Created large ZIP archive"
101
102
103 # ============================================
104 # FILE 8: MANY SMALL TEXT FILES
105 # ============================================
106
107 echo ""
108 echo "Creating 500 small text files..."
109
110 for i in {1..500}; do
111
112     sudo bash -c "echo 'UNIQUE_SIGNATURE_DS${ITER}_SMALL_${i}' > '
    $ITER_TEST_DIR/tiny_${i}.txt'"
113
114 done
115
116 echo "Created 500 tiny text files"
117
118
119 # ============================================
120 # FILE 9: VALID JPEG IMAGE
121 # ============================================
122
123 echo ""
124 echo "Creating JPEG file..."
125
126 # Minimal valid 1x1 JPEG image.
127 JPEG_BASE64="/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAP
    //////////////////////////////////
```

```
128 ////////////////////////////////////////////////////2wBDAf
        //////////////////////
129 ////////////////////////////////////////////////////////////////
        wAARCAABAAEDASI
130 AAhEBAxEB/8QAFQABAQAAAAAAAAAAAAAAAAAAAAX/
        xAAUEAEAAAAAAAAAAAAAAAAAAAAA/9oADAMBAAI
131 QAxAAAAH/AP/EABQQAQAAAAAAAAAAAAAAAAAACD/2gAIAQEAAT8Af//
        EABQRAQAAAAAAAAAAAAAAAA
132 AABD/2gAIAQIBAT8Af//EABQRAQAAAAAAAAAAAAAAAAAABD/2gAIAQMBAT8Af//Z"
133
134 echo "$JPEG_BASE64" | base64 -d | sudo tee "$ITER_TEST_DIR/image.
        jpg" > /dev/null
135
136 # Add a dataset-specific text signature after the JPEG.
137 # This does not affect the JPEG header and gives us a searchable
138 # experiment-specific marker.
139 echo "UNIQUE_SIGNATURE_DS${ITER}_IMAGE" | \
140     sudo tee -a "$ITER_TEST_DIR/image.jpg" > /dev/null
141
142 echo "image.jpg created"
143
144
145 # ============================================
146 # FILE 10: VALID PDF
147 # ============================================
148
149 echo ""
150 echo "Creating PDF file..."
151
152 sudo bash -c "cat > '$ITER_TEST_DIR/document.pdf' << ENDPDF
153 %PDF-1.4
154 1 0 obj
155 << /Type /Catalog /Pages 2 0 R >>
156 endobj
157 2 0 obj
158 << /Type /Pages /Kids [3 0 R] /Count 1 >>
159 endobj
160 3 0 obj
161 << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0
        R >>
162 endobj
163 4 0 obj
164 << /Length 120 >>
165 stream
166 BT
167 /F1 24 Tf
168 100 700 Td
169 (UNIQUE_SIGNATURE_DS${ITER}_PDF) Tj
170 0 -40 Td
```

```
(Experimental secure deletion dataset) Tj
ET
endstream
endobj
xref
0 5
0000000000 65535 f
0000000009 00000 n
0000000058 00000 n
0000000115 00000 n
0000000210 00000 n
trailer
<< /Size 5 /Root 1 0 R >>
startxref
380
%%EOF
ENDPDF"

echo "document.pdf created"

# ============================================
# FILE 11: CHECKSUMS
# ============================================

echo ""
echo "Computing MD5 checksums..."

sudo bash -c "find '$ITER_TEST_DIR' -maxdepth 1 -type f \
    ! -name 'checksums_${ITER}.md5' \
    -exec md5sum {} \; | sort > '$ITER_TEST_DIR/checksums_${ITER}.
   md5'"

echo "MD5 checksums saved"

# ============================================
# DATASET SIZE / FILE COUNT
# ============================================

FILE_COUNT=$(sudo find "$ITER_TEST_DIR" -type f | wc -l)
DATASET_SIZE=$(sudo du -sh "$ITER_TEST_DIR" | cut -f1)

echo ""
echo "========================================="
echo "DATASET ${ITER} CREATED"
echo "========================================="
echo "Location:     $ITER_TEST_DIR"
echo "Dataset size: $DATASET_SIZE"
echo "File count:   $FILE_COUNT"
echo "========================================="
```

```
219
220 # ============================================
221 # PHASE 4: Delete Test Data
222 # ============================================
223
224 echo ""
225 echo "=== [ITER $ITER - PHASE 4]: Delete Test Data ==="
226
227 BEFORE=$(find "$ITER_TEST_DIR" -type f | wc -l)
228 echo "Deleting $BEFORE test files..."
229 sudo rm -rf "$ITER_TEST_DIR"/*
230 sync
231 sleep 5
232 echo "Test data deleted"
233
234
235
236 # ============================================
237 # PHASE 5: Measure TRIM Latency
238 # ============================================
239
240 echo ""
241 echo "=== [ITER $ITER - PHASE 5]: Measure TRIM Latency ==="
242
243
244     # Clear caches
245     sync && echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
246
247     # Run fstrim with timing
248     LOG_FILE="$LOG_DIR/trim_${FILL_LEVEL}_iter${ITER}_$(date +%H%M
    %S).log"
249     /usr/bin/time -v sudo fstrim -v "$MOUNT_POINT" 2>&1 | tee "
    $LOG_FILE"
250
251     # Extract timing
252     REAL_TIME=$(grep "Elapsed" "$LOG_FILE" | awk '{print $NF}')
253     USER_TIME=$(grep "User time" "$LOG_FILE" | awk '{print $NF}')
254     SYS_TIME=$(grep "System time" "$LOG_FILE" | awk '{print $NF}')
255     TRIMMED=$(grep -o "[0-9]* bytes" "$LOG_FILE" | tail -1 | sed '
    s/ bytes//')
256     echo "    Real: ${REAL_TIME}s, Trimmed: ${TRIMMED}B"
257
258
259
260 # ============================================
261 # PHASE 6: Attempt Recovery at CURRENT_WAIT
262 # ============================================
263
264 echo ""
```

```
265 echo "=== [ITER $ITER - PHASE 6]: Recovery Attempts ==="
266 echo "  Waiting ${CURRENT_WAIT} minutes before recovery..."
267
268
269     # Wait the specified time
270     if [ $CURRENT_WAIT -gt 0 ]; then
271         echo "    Waiting ${CURRENT_WAIT} minutes..."
272         sleep $((CURRENT_WAIT * 60))
273     fi
274
275     # Try recovery with foremost
276     RECOVERY_DIR="$LOG_DIR/recovered_${FILL_LEVEL}_iter${ITER}_${
    CURRENT_WAIT}min"
277     mkdir -p "$RECOVERY_DIR"
278
279     # Use foremost to recover deleted files (quick)
280     echo "    Running foremost analysis..."
281     sudo foremost -i /dev/sda1 -o "$RECOVERY_DIR" -t all
282
283     # Count recovered files (simplified - check if any files
    appear)
284     RECOVERED=0
285     if [ -d "$RECOVERY_DIR" ]; then
286         RECOVERED=$(find "$RECOVERY_DIR" -type f ! -name "audit.
    txt" 2>/dev/null | wc -l)
287     fi
288
289     echo "    Recovered: $RECOVERED files"
290
291 # Capture timestamp for this run
292     RUN_TIMESTAMP=$(date +%Y%m%d_%H%M%S)
293
294
295 echo "$RUN_TIMESTAMP,$ITER,$CURRENT_WAIT,$FILL_LEVEL,$REAL_TIME,
    $USER_TIME,$SYS_TIME,$TRIMMED,$RECOVERED" >> $LOG_DIR/
    experiment_results.csv
296
297
298     echo ""
299     echo "Iteration $ITER complete (waited ${CURRENT_WAIT} minutes
    )"
300     echo "---"
301
302     # Small delay between iterations
303     sleep 2
304 done
```

Listing A.4: PHASE 3-6: Main Experiment Loop